

(HEADER 1)

📁 ai_agent/ · 📁 llm_gateway/ · 📁 mcp_server/ · 📁 evals_framework/



agentic-ai:

1. What It Does (Plain English & Analogy)

agentic-ai is the central autonomous agent platform and multi-provider AI hub orchestrating ReAct loops, FastMCP tools, vector memory, multi-agent debate swarms, and enterprise gateway governance.

 **Analogy:** Like the central brain and nervous system of an autonomous robot: processing sensory inputs (tools and memory), deliberating actions, coordinating limbs, and learning from experience.

2. Why & How It Helps (Value Proposition)

Solves the fragmentation of building agentic systems by unifying LLM proxies, tool execution, persistent memory, and evaluation benchmarking into a single production-ready architecture.

Table of Contents · Architecture Modules

 AI Agent	9 Subsystems	 Evals Framework	7 Subsystems
 Scripts & Tools	3 Subsystems	 Workspace	2 Subsystems
 LLM Gateway	12 Subsystems	 MCP Server	4 Subsystems
 Memory Store	2 Subsystems	 Web UI Studio	3 Subsystems

SUB HEADER (OF HEADER 1):

`ai_agent/agent.py`



AI Agent

1. What It Does (Plain English & Analogy)

The AI Agent module runs the autonomous ReAct reasoning loop, dispatches external tools, coordinates multi-agent debate tribunals, and executes parallel swarm workflows.



Analogy: Like a master detective directing an investigative squad: gathering clues from witnesses (tools), consulting case files (memory), and weighing theories with peers before drawing conclusions.

2. Why & How It Helps (Value Proposition)

Transitions AI from static text generators into autonomous problem solvers capable of executing multi-step business logic, self-correcting errors, and verifying external facts.

SUB HEADER (OF AI AGENT):

`ai_agent/agent.py`

Core ReAct Engine

1. What It Does (Plain English & Analogy)

Core ReAct Engine is a foundational architectural component of AI Agent that iterative reasoning loop with tool-rag & state hydration.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the AI Agent pipeline.

COMPONENT (UNDER CORE REACT ENGINE):

`ai_agent/agent.py:AgentRunResult`

AgentRunResult

1. What It Does (Plain English & Analogy)

AgentRunResult is a foundational architectural component of Core ReAct Engine that execution outcome schema and token telemetry.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the AI Agent pipeline.

ATOMIC FEATURE:

response: str

1. What It Does (Plain English & Analogy)

response: str is a foundational architectural component of AgentRunResult that synthesized markdown or text answer.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

tool_calls_executed: list

1. What It Does (Plain English & Analogy)

tool_calls_executed: list is a foundational architectural component of AgentRunResult that log of all successfully called tools.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

total_prompt_tokens: int

1. What It Does (Plain English & Analogy)

total_prompt_tokens: int is a foundational architectural component of AgentRunResult that gateway prompt token accumulator.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

total_completion_tokens: int

1. What It Does (Plain English & Analogy)

total_completion_tokens: int is a foundational architectural component of AgentRunResult that gateway completion token accumulator.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

active_skills: list

1. What It Does (Plain English & Analogy)

active_skills: list is an expert persona module that skills loaded for this turn.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER CORE REACT ENGINE):

`ai_agent/agent.py:run()`

ReAct Loop Algorithm

1. What It Does (Plain English & Analogy)

ReAct Loop Algorithm is a foundational architectural component of Core ReAct Engine that max 6-iteration reason-act loop with deduplication & fallbacks.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

max 6 iterations guard

1. What It Does (Plain English & Analogy)

max 6 iterations guard is an automated safety defense mechanism that prevents runaway infinite reasoning loops.



Analogy: Like a circuit breaker in an electrical panel that trips the instant excess voltage is detected to prevent a fire.

2. Why & How It Helps (Value Proposition)

Guarantees fail-safe protection against infinite reasoning loops, malicious prompt injections, and accidental credential leaks.

ATOMIC FEATURE:

text-based call recovery

1. What It Does (Plain English & Analogy)

text-based call recovery is a schema resilience utility that heuristic parser for malformed tool json.



Analogy: Like an intelligent spelling auto-correct system on a smartphone keyboard that fixes minor typos seamlessly.

2. Why & How It Helps (Value Proposition)

Prevents preventable tool call crashes and runtime validation errors caused by minor LLM argument naming discrepancies.

ATOMIC FEATURE:

dedup & loop breaker

1. What It Does (Plain English & Analogy)

dedup & loop breaker is an automated safety defense mechanism that detects identical consecutive tool arguments.



Analogy: Like a circuit breaker in an electrical panel that trips the instant excess voltage is detected to prevent a fire.

2. Why & How It Helps (Value Proposition)

Guarantees fail-safe protection against infinite reasoning loops, malicious prompt injections, and accidental credential leaks.

ATOMIC FEATURE:

SHA-256 idempotency cache

1. What It Does (Plain English & Analogy)

SHA-256 idempotency cache is a high-speed caching mechanism that caches tool results to avoid duplicate work.



Analogy: Like keeping a carbon copy of today's verified manifest on your desk so you don't have to telephone the warehouse twice for the same order.

2. Why & How It Helps (Value Proposition)

Slashes response latency to zero on repeated queries and prevents duplicate side effects on external services.

ATOMIC FEATURE:

firewall output sanitization

1. What It Does (Plain English & Analogy)

firewall output sanitization is an automated safety defense mechanism that scans tool outputs for prompt injections.



Analogy: Like a circuit breaker in an electrical panel that trips the instant excess voltage is detected to prevent a fire.

2. Why & How It Helps (Value Proposition)

Guarantees fail-safe protection against infinite reasoning loops, malicious prompt injections, and accidental credential leaks.

ATOMIC FEATURE:

fallback synthesis

1. What It Does (Plain English & Analogy)

fallback synthesis is a foundational architectural component of ReAct Loop Algorithm that graceful answer generation if iteration limit reached.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER CORE REACT ENGINE):

```
ai_agent/agent.py:filter_tools()
```

Tool-RAG Filter

1. What It Does (Plain English & Analogy)

Tool-RAG Filter is a foundational architectural component of Core ReAct Engine that semantic ranking of tools to stay within context limits.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

query token scoring

1. What It Does (Plain English & Analogy)

query token scoring is a foundational architectural component of Tool-RAG Filter that bm25 keyword overlap against tool descriptions.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

active skill boost

1. What It Does (Plain English & Analogy)

active skill boost is an expert persona module that pins tools matching currently loaded persona.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

ATOMIC FEATURE:

intent detection

1. What It Does (Plain English & Analogy)

intent detection is a foundational architectural component of Tool-RAG Filter that heuristics for math, web, code, file intent.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

essential tools retention

1. What It Does (Plain English & Analogy)

essential tools retention is a foundational architectural component of Tool-RAG Filter that always includes calculator & memory recall.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

max 12 tools cap

1. What It Does (Plain English & Analogy)

max 12 tools cap is a foundational architectural component of Tool-RAG Filter that preserves llm attention window.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER CORE REACT ENGINE): `ai_agent/agent.py`

Progressive Disclosure

1. What It Does (Plain English & Analogy)

Progressive Disclosure is a foundational architectural component of Core ReAct Engine that on-demand dynamic skill discovery & prompt injection.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

`discover_skills` tool

1. What It Does (Plain English & Analogy)

`discover_skills` tool is an expert persona module that lists available domain personas.

 **Analogy:** Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

ATOMIC FEATURE:

`load_skill` tool

1. What It Does (Plain English & Analogy)

`load_skill` tool is an expert persona module that loads full system instructions for a persona.

 **Analogy:** Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

ATOMIC FEATURE:

activate_skill hook

1. What It Does (Plain English & Analogy)

activate_skill hook is an expert persona module that injects persona rules into prompt template.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

ATOMIC FEATURE:

85% token reduction

1. What It Does (Plain English & Analogy)

85% token reduction is a foundational architectural component of Progressive Disclosure that loads instructions only when requested.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER CORE REACT ENGINE):

```
ai_agent/agent.py:hydrate()
```

Durable State Engine

1. What It Does (Plain English & Analogy)

Durable State Engine is a foundational architectural component of Core ReAct Engine that sqlite session persistence with turn hydration.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

ATOMIC FEATURE:

| `_hydrate_session()`

1. What It Does (Plain English & Analogy)

`_hydrate_session()` is a foundational architectural component of Durable State Engine that loads conversation history from sqlite.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

| `_persist_turn()`

1. What It Does (Plain English & Analogy)

`_persist_turn()` is a foundational architectural component of Durable State Engine that saves user & assistant messages.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

| `agent_sessions` table

1. What It Does (Plain English & Analogy)

`agent_sessions` table is a foundational architectural component of Durable State Engine that tracks active conversation metadata.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

agent_turns table

1. What It Does (Plain English & Analogy)

agent_turns table is a foundational architectural component of Durable State Engine that stores message payloads and token stats.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

SUB HEADER (OF 🧠 AI AGENT):

ai_agent/gateway_client.py

Gateway Client

1. What It Does (Plain English & Analogy)

Gateway Client is a foundational architectural component of 🧠 AI Agent that dual-transport client communicating with llm gateway.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER GATEWAY CLIENT):

HTTP transport (httpx.AsyncClient)

1. What It Does (Plain English & Analogy)

HTTP transport (httpx.AsyncClient) is a foundational architectural component of Gateway Client that high-concurrency async rest client.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER GATEWAY CLIENT):**Stdio subprocess IPC****1. What It Does (Plain English & Analogy)**

Stdio subprocess IPC is a foundational architectural component of Gateway Client that zero-network fallback transport.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER GATEWAY CLIENT):**chat_completion() method****1. What It Does (Plain English & Analogy)**

chat_completion() method is a foundational architectural component of Gateway Client that routes prompts through litellm.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER GATEWAY CLIENT):**check_health() endpoint****1. What It Does (Plain English & Analogy)**

check_health() endpoint is a foundational architectural component of Gateway Client that gateway connectivity probe.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER GATEWAY CLIENT):

| list_models() endpoint

1. What It Does (Plain English & Analogy)

list_models() endpoint is a configured LLM endpoint that provides queries active model catalog.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

COMPONENT (UNDER GATEWAY CLIENT):

| get_audit_logs() endpoint

1. What It Does (Plain English & Analogy)

get_audit_logs() endpoint is a foundational architectural component of Gateway Client that fetches recent gateway telemetry.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚢 AI Agent pipeline.

COMPONENT (UNDER GATEWAY CLIENT):

| get_gateway_stats() endpoint

1. What It Does (Plain English & Analogy)

get_gateway_stats() endpoint is a foundational architectural component of Gateway Client that p50/p90/p99 latency metrics.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚢 AI Agent pipeline.

COMPONENT (UNDER GATEWAY CLIENT):

X-Caller-Id tracing header

1. What It Does (Plain English & Analogy)

X-Caller-Id tracing header is a foundational architectural component of Gateway Client that passes caller identity for rate-limiting.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

SUB HEADER (OF 🧠 AI AGENT):

```
ai_agent/mcp_client.py
```

MCP Client

1. What It Does (Plain English & Analogy)

MCP Client is a foundational architectural component of 🧠 AI Agent that fastmcp integration client with auto-healing and normalization.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER MCP CLIENT):

stdio_client subprocess spawn

1. What It Does (Plain English & Analogy)

stdio_client subprocess spawn is a foundational architectural component of MCP Client that launches mcp_server on demand.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER MCP CLIENT):**| list_tools_for_llm()****1. What It Does (Plain English & Analogy)**

list_tools_for_llm() is a foundational architectural component of MCP Client that converts fastmcp schemas to openai tool definitions.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER MCP CLIENT):**| list_skills() discovery****1. What It Does (Plain English & Analogy)**

list_skills() discovery is an expert persona module that fetches metadata for all 10 domain skills.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER MCP CLIENT):**| get_skill_prompt() loader****1. What It Does (Plain English & Analogy)**

get_skill_prompt() loader is an expert persona module that fetches system prompt for selected skill.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER MCP CLIENT):

execute_tool() dispatcher

1. What It Does (Plain English & Analogy)

execute_tool() dispatcher is a foundational architectural component of MCP Client that runs tool with timeout and error handling.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER MCP CLIENT):

Argument Normalization

1. What It Does (Plain English & Analogy)

Argument Normalization is a foundational architectural component of MCP Client that maps llm hallucinated param names to schemas.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

calculator aliases (expr, equation -> expression)

1. What It Does (Plain English & Analogy)

calculator aliases (expr, equation -> expression) is an exact mathematical computation tool that math param normalization.



Analogy: Like an accountant's desktop adding machine that computes totals and percentages with 100% mathematical precision without rounding errors.

2. Why & How It Helps (Value Proposition)

Prevents LLM math hallucinations on financial totals, split bills, and arithmetic expressions by delegating numbers to deterministic code.

ATOMIC FEATURE:

weather aliases (place, loc -> city)

1. What It Does (Plain English & Analogy)

weather aliases (place, loc -> city) is a schema resilience utility that location param normalization.



Analogy: Like an intelligent spelling auto-correct system on a smartphone keyboard that fixes minor typos seamlessly.

2. Why & How It Helps (Value Proposition)

Prevents preventable tool call crashes and runtime validation errors caused by minor LLM argument naming discrepancies.

ATOMIC FEATURE:

workspace aliases (path, file -> filename)

1. What It Does (Plain English & Analogy)

workspace aliases (path, file -> filename) is a schema resilience utility that file param normalization.



Analogy: Like an intelligent spelling auto-correct system on a smartphone keyboard that fixes minor typos seamlessly.

2. Why & How It Helps (Value Proposition)

Prevents preventable tool call crashes and runtime validation errors caused by minor LLM argument naming discrepancies.

ATOMIC FEATURE:

tip/split aliases (amount, bill -> total)

1. What It Does (Plain English & Analogy)

tip/split aliases (amount, bill -> total) is a schema resilience utility that bill param normalization.



Analogy: Like an intelligent spelling auto-correct system on a smartphone keyboard that fixes minor typos seamlessly.

2. Why & How It Helps (Value Proposition)

Prevents preventable tool call crashes and runtime validation errors caused by minor LLM argument naming discrepancies.

COMPONENT (UNDER MCP CLIENT):

self-healing reconnect

1. What It Does (Plain English & Analogy)

self-healing reconnect is a foundational architectural component of MCP Client that auto-restarts subprocess on exit or crash.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

SUB HEADER (OF 🧠 AI AGENT):

📁 ai_agent/task_planner.py

Task Planner (DAG)

1. What It Does (Plain English & Analogy)

Task Planner (DAG) is an expert persona module that decomposes complex goals into acyclic execution graphs.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER TASK PLANNER (DAG)):

SubTask Pydantic model

1. What It Does (Plain English & Analogy)

SubTask Pydantic model is an asynchronous workflow coordination component that task_id, description, skill, depends_on, status.



Analogy: Like an airport ground crew coordinator scheduling baggage loading, refueling, and catering so all teams work in parallel without blocking the runway.

2. Why & How It Helps (Value Proposition)

Unlocks high-concurrency parallel task execution while enforcing strict topological ordering and crash recovery.

COMPONENT (UNDER TASK PLANNER (DAG)):**TaskDAG container****1. What It Does (Plain English & Analogy)**

TaskDAG container is an asynchronous workflow coordination component that graph structure with validation and execution queues.



Analogy: Like an airport ground crew coordinator scheduling baggage loading, refueling, and catering so all teams work in parallel without blocking the runway.

2. Why & How It Helps (Value Proposition)

Unlocks high-concurrency parallel task execution while enforcing strict topological ordering and crash recovery.

COMPONENT (UNDER TASK PLANNER (DAG)):**Kahn's algorithm validation****1. What It Does (Plain English & Analogy)**

Kahn's algorithm validation is a foundational architectural component of Task Planner (DAG) that verifies acyclicity before execution starts.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER TASK PLANNER (DAG)):**infer_skill heuristics****1. What It Does (Plain English & Analogy)**

infer_skill heuristics is an expert persona module that matches task text to relevant domain skill.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER TASK PLANNER (DAG)):**| build_decomposition_prompt()****1. What It Does (Plain English & Analogy)**

build_decomposition_prompt() is a foundational architectural component of Task Planner (DAG) that few-shot prompt for llm planning.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER TASK PLANNER (DAG)):**| parse_decomposition_response()****1. What It Does (Plain English & Analogy)**

parse_decomposition_response() is a foundational architectural component of Task Planner (DAG) that extracts json task list from llm answer.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER TASK PLANNER (DAG)):**| cycle_repair_fallback****1. What It Does (Plain English & Analogy)**

cycle_repair_fallback is a foundational architectural component of Task Planner (DAG) that removes back-edges automatically if cycle found.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

SUB HEADER (OF 🧠 AI AGENT): `ai_agent/orchestrator.py`

Orchestrator Swarm

1. What It Does (Plain English & Analogy)

Orchestrator Swarm is an asynchronous workflow coordination component that supervisor agent managing parallel worker execution.



Analogy: Like an airport ground crew coordinator scheduling baggage loading, refueling, and catering so all teams work in parallel without blocking the runway.

2. Why & How It Helps (Value Proposition)

Unlocks high-concurrency parallel task execution while enforcing strict topological ordering and crash recovery.

COMPONENT (UNDER ORCHESTRATOR SWARM):

SupervisorAgent coordinator

1. What It Does (Plain English & Analogy)

SupervisorAgent coordinator is a foundational architectural component of Orchestrator Swarm that central task dispatcher and synthesizer.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER ORCHESTRATOR SWARM):

decompose prompt method

1. What It Does (Plain English & Analogy)

decompose prompt method is a foundational architectural component of Orchestrator Swarm that prompts planner for subtasks.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER ORCHESTRATOR SWARM):**| _execute_worker() runner****1. What It Does (Plain English & Analogy)**

_execute_worker() runner is an asynchronous workflow coordination component that executes isolated worker for each task.



Analogy: Like an airport ground crew coordinator scheduling baggage loading, refueling, and catering so all teams work in parallel without blocking the runway.

2. Why & How It Helps (Value Proposition)

Unlocks high-concurrency parallel task execution while enforcing strict topological ordering and crash recovery.

COMPONENT (UNDER ORCHESTRATOR SWARM):**| _execute_dag() topological runner****1. What It Does (Plain English & Analogy)**

_execute_dag() topological runner is a foundational architectural component of Orchestrator Swarm that executes independent tasks concurrently.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER ORCHESTRATOR SWARM):**| _synthesize results step****1. What It Does (Plain English & Analogy)**

_synthesize results step is a foundational architectural component of Orchestrator Swarm that aggregates all task outputs into final report.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER ORCHESTRATOR SWARM):

self-healing retry logic

1. What It Does (Plain English & Analogy)

self-healing retry logic is a foundational architectural component of Orchestrator Swarm that retries failed subtasks with alternative tools.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER ORCHESTRATOR SWARM):

semaphore concurrency limiter

1. What It Does (Plain English & Analogy)

semaphore concurrency limiter is a foundational architectural component of Orchestrator Swarm that limits simultaneous workers to avoid rate limits.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER ORCHESTRATOR SWARM):

deadlock resolution

1. What It Does (Plain English & Analogy)

deadlock resolution is a foundational architectural component of Orchestrator Swarm that detects and unblocks stuck tasks.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER ORCHESTRATOR SWARM):

node_checkpoints SQLite logging

1. What It Does (Plain English & Analogy)

node_checkpoints SQLite logging is a foundational architectural component of Orchestrator Swarm that stores incremental state for crash recovery.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧑‍🚀 AI Agent pipeline.

SUB HEADER (OF 🧑‍🚀 AI AGENT):

📁 ai_agent/debate.py

Adversarial Debate

1. What It Does (Plain English & Analogy)

Adversarial Debate is a foundational architectural component of 🧑‍🚀 AI Agent that multi-agent peer review with risk arbitration.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧑‍🚀 AI Agent pipeline.

COMPONENT (UNDER ADVERSARIAL DEBATE):

MultiAgentDebateManager

1. What It Does (Plain English & Analogy)

MultiAgentDebateManager is a foundational architectural component of Adversarial Debate that orchestrates multi-perspective evaluation.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧑‍🚀 AI Agent pipeline.

COMPONENT (UNDER ADVERSARIAL DEBATE):**Proposer Agent****1. What It Does (Plain English & Analogy)**

Proposer Agent is a foundational architectural component of Adversarial Debate that drafts initial solution or strategy.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚢 AI Agent pipeline.

COMPONENT (UNDER ADVERSARIAL DEBATE):**Red-Team Critic Agent****1. What It Does (Plain English & Analogy)**

Red-Team Critic Agent is a foundational architectural component of Adversarial Debate that identifies risks, flaws, edge-cases, vulnerabilities.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚢 AI Agent pipeline.

COMPONENT (UNDER ADVERSARIAL DEBATE):**Arbitrator Agent****1. What It Does (Plain English & Analogy)**

Arbitrator Agent is a foundational architectural component of Adversarial Debate that synthesizes consensus and issues verdict.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚢 AI Agent pipeline.

COMPONENT (UNDER ADVERSARIAL DEBATE):

RISK_SCORE extraction (0-100)

1. What It Does (Plain English & Analogy)

RISK_SCORE extraction (0-100) is a foundational architectural component of Adversarial Debate that quantified safety assessment.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚢 AI Agent pipeline.

COMPONENT (UNDER ADVERSARIAL DEBATE):

confidence scoring

1. What It Does (Plain English & Analogy)

confidence scoring is a foundational architectural component of Adversarial Debate that probabilistic agreement metric.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚢 AI Agent pipeline.

COMPONENT (UNDER ADVERSARIAL DEBATE):

distinct models per role

1. What It Does (Plain English & Analogy)

distinct models per role is a configured LLM endpoint that provides e.g. qwen for logic, gemma for drafting.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

SUB HEADER (OF 🧠 AI AGENT): `ai_agent/federation.py`

Federated MCP

1. What It Does (Plain English & Analogy)

Federated MCP is a foundational architectural component of 🧠 AI Agent that connects to multiple independent tool servers.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER FEDERATED MCP):

register_server()

1. What It Does (Plain English & Analogy)

register_server() is a foundational architectural component of Federated MCP that adds server endpoint to federation pool.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER FEDERATED MCP):

connect_all() async init

1. What It Does (Plain English & Analogy)

connect_all() async init is a foundational architectural component of Federated MCP that spawns all connections concurrently.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER FEDERATED MCP):

refresh_tools_catalog()

1. What It Does (Plain English & Analogy)

refresh_tools_catalog() is a foundational architectural component of Federated MCP that merges tool definitions from all servers.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER FEDERATED MCP):

namespace tagging

1. What It Does (Plain English & Analogy)

namespace tagging is a foundational architectural component of Federated MCP that prefixes tool names (e.g. servera_read_file).



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER FEDERATED MCP):

routed execution

1. What It Does (Plain English & Analogy)

routed execution is a foundational architectural component of Federated MCP that routes tool call to origin server.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧰 AI Agent pipeline.

COMPONENT (UNDER FEDERATED MCP):

STDIO + SSE transport support

1. What It Does (Plain English & Analogy)

STDIO + SSE transport support is a foundational architectural component of Federated MCP that connects to local scripts or remote servers.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

SUB HEADER (OF 🧠 AI AGENT): `ai_agent/router.py`

16 REST API Endpoints

1. What It Does (Plain English & Analogy)

16 REST API Endpoints is a foundational architectural component of 🧠 AI Agent that fastapi router mounted directly into the llm gateway.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER 16 REST API ENDPOINTS):

💬 Chat & Streaming APIs

1. What It Does (Plain English & Analogy)

💬 Chat & Streaming APIs is a foundational architectural component of 16 REST API Endpoints that interactive conversational endpoints.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

POST /api/chat

1. What It Does (Plain English & Analogy)

POST /api/chat is an authenticated HTTP REST endpoint providing programmatic access to single-turn react completion.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

POST /api/chat/stream

1. What It Does (Plain English & Analogy)

POST /api/chat/stream is an authenticated HTTP REST endpoint providing programmatic access to sse token and tool execution stream.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

POST /api/chat/clear

1. What It Does (Plain English & Analogy)

POST /api/chat/clear is an authenticated HTTP REST endpoint providing programmatic access to resets session conversation history.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 16 REST API ENDPOINTS):

■ Canvas & Pipeline APIs

1. What It Does (Plain English & Analogy)

■ Canvas & Pipeline APIs is a foundational architectural component of 16 REST API Endpoints that 2d visual workflow pipeline endpoints.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧑‍🤖 AI Agent pipeline.

ATOMIC FEATURE:

POST /api/canvas/execute

1. What It Does (Plain English & Analogy)

POST /api/canvas/execute is an authenticated HTTP REST endpoint providing programmatic access to executes visual dag pipeline.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

POST /api/canvas/resume/{id}

1. What It Does (Plain English & Analogy)

POST /api/canvas/resume/{id} is an authenticated HTTP REST endpoint providing programmatic access to resumes failed pipeline from checkpoint.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:**GET /api/canvas/runs****1. What It Does (Plain English & Analogy)**

GET /api/canvas/runs is an authenticated HTTP REST endpoint providing programmatic access to fetches pipeline run history.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:**POST /api/canvas/pipelines****1. What It Does (Plain English & Analogy)**

POST /api/canvas/pipelines is an authenticated HTTP REST endpoint providing programmatic access to saves reusable dag template.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:**DELETE /api/canvas/pipelines/{id}****1. What It Does (Plain English & Analogy)**

DELETE /api/canvas/pipelines/{id} is an authenticated HTTP REST endpoint providing programmatic access to removes saved pipeline template.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 16 REST API ENDPOINTS):

Swarm & Debate APIs

1. What It Does (Plain English & Analogy)

 Swarm & Debate APIs is an asynchronous workflow coordination component that multi-agent coordination endpoints.

 **Analogy:** Like an airport ground crew coordinator scheduling baggage loading, refueling, and catering so all teams work in parallel without blocking the runway.

2. Why & How It Helps (Value Proposition)

Unlocks high-concurrency parallel task execution while enforcing strict topological ordering and crash recovery.

ATOMIC FEATURE:

POST /api/orchestrator/run

1. What It Does (Plain English & Analogy)

POST /api/orchestrator/run is an authenticated HTTP REST endpoint providing programmatic access to launches full multi-agent dag run.

 **Analogy:** Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

POST /api/orchestrator/run-stream

1. What It Does (Plain English & Analogy)

POST /api/orchestrator/run-stream is an authenticated HTTP REST endpoint providing programmatic access to streams dag worker events.

 **Analogy:** Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:**GET /api/orchestrator/runs****1. What It Does (Plain English & Analogy)**

GET /api/orchestrator/runs is an authenticated HTTP REST endpoint providing programmatic access to lists all historical orchestration runs.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:**POST /api/debate****1. What It Does (Plain English & Analogy)**

POST /api/debate is an authenticated HTTP REST endpoint providing programmatic access to executes 3-agent adversarial debate.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 16 REST API ENDPOINTS):**Session & State APIs****1. What It Does (Plain English & Analogy)**

Session & State APIs is a foundational architectural component of 16 REST API Endpoints that conversation state query endpoints.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

ATOMIC FEATURE:

GET /api/chat/sessions/{id}

1. What It Does (Plain English & Analogy)

GET /api/chat/sessions/{id} is an authenticated HTTP REST endpoint providing programmatic access to fetches messages for specific session.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

GET /api/chat/sessions

1. What It Does (Plain English & Analogy)

GET /api/chat/sessions is an authenticated HTTP REST endpoint providing programmatic access to lists all active sessions.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

SUB HEADER (OF 🧠 AI AGENT):

```
ai_agent/cli.py
```

Rich Terminal CLI

1. What It Does (Plain English & Analogy)

Rich Terminal CLI is a foundational architectural component of 🧠 AI Agent that interactive developer cli with syntax highlighting.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER RICH TERMINAL CLI):**| /skills command****1. What It Does (Plain English & Analogy)**

/skills command is an expert persona module that displays available skill cards.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER RICH TERMINAL CLI):**| /skill activate <name>****1. What It Does (Plain English & Analogy)**

/skill activate <name> is an expert persona module that switches agent active persona.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER RICH TERMINAL CLI):**| /stats command****1. What It Does (Plain English & Analogy)**

/stats command is a foundational architectural component of Rich Terminal CLI that renders gateway telemetry table.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧩 AI Agent pipeline.

COMPONENT (UNDER RICH TERMINAL CLI):

/logs command

1. What It Does (Plain English & Analogy)

/logs command is a foundational architectural component of Rich Terminal CLI that dumps recent conversation turns.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

COMPONENT (UNDER RICH TERMINAL CLI):

Rich Markdown streaming

1. What It Does (Plain English & Analogy)

Rich Markdown streaming is a foundational architectural component of Rich Terminal CLI that syntax-highlighted terminal output.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🧠 AI Agent pipeline.

SUB HEADER (OF HEADER 1): `evals_framework/runner.py`

Evals Framework

1. What It Does (Plain English & Analogy)

The Evals Framework runs automated multi-model benchmark evaluations with 9 specialized graders, producing quantitative scorecards across accuracy, reasoning, safety, and efficiency.

 **Analogy:** Like a collegiate accreditation board: putting students through standardized, rigorous examinations across multiple academic subjects to grade real competence.

2. Why & How It Helps (Value Proposition)

Replaces guessing and "prompt vibes" with scientific, reproducible metrics, instantly flagging regressions in tool selection, factual hallucinations, or PII leaks before shipping.

SUB HEADER (OF  EVALS FRAMEWORK): `evals_framework/runner.py`

Benchmark Runner Engine

1. What It Does (Plain English & Analogy)

Benchmark Runner Engine is an evaluation grading component that orchestrates multi-model evaluations and weighted scoring.

 **Analogy:** Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER BENCHMARK RUNNER ENGINE):

`load_test_cases()`

1. What It Does (Plain English & Analogy)

`load_test_cases()` is an evaluation grading component that reads benchmark json datasets.

 **Analogy:** Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER BENCHMARK RUNNER ENGINE):**| run_suite() dispatcher****1. What It Does (Plain English & Analogy)**

run_suite() dispatcher is an evaluation grading component that executes agent on test suite with retry.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER BENCHMARK RUNNER ENGINE):**| multi-iteration averaging****1. What It Does (Plain English & Analogy)**

multi-iteration averaging is an evaluation grading component that reduces variance by running n passes.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER BENCHMARK RUNNER ENGINE):**| Composite Weighted Scoring****1. What It Does (Plain English & Analogy)**

Composite Weighted Scoring is an evaluation grading component that formula: sum of grader scores * weights (100% total).



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Accuracy & Code (30%)

1. What It Does (Plain English & Analogy)

 Accuracy & Code (30%) is an evaluation grading component that tool calling, syntax and relevance.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Deterministic Grader (20%)

1. What It Does (Plain English & Analogy)

Deterministic Grader (20%) is an evaluation grading component that tool presence, arguments, and regex.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Code Syntax Grader (5%)

1. What It Does (Plain English & Analogy)

Code Syntax Grader (5%) is an evaluation grading component that ast parsing and security validation.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Relevance Grader (5%)

1. What It Does (Plain English & Analogy)

Relevance Grader (5%) is an evaluation grading component that fluff and verbosity penalty.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:



Reasoning & Grounding (40%)

1. What It Does (Plain English & Analogy)

Reasoning & Grounding (40%) is an evaluation grading component that logical correctness and truthfulness.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

LLM Judge (15%)

1. What It Does (Plain English & Analogy)

LLM Judge (15%) is an evaluation grading component that intent adherence & politeness.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Faithfulness (15%)

1. What It Does (Plain English & Analogy)

Faithfulness (15%) is an evaluation grading component that atomic assertion ground truth check.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Fact Checker (10%)

1. What It Does (Plain English & Analogy)

Fact Checker (10%) is an evaluation grading component that hallucination detection against tools.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Efficiency, Safety & Style (30%)

1. What It Does (Plain English & Analogy)

Efficiency, Safety & Style (30%) is an evaluation grading component that resource optimization and guardrails.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Efficiency Grader (15%)

1. What It Does (Plain English & Analogy)

Efficiency Grader (15%) is an evaluation grading component that token budget, latency, loop count.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Safety Grader (10%)

1. What It Does (Plain English & Analogy)

Safety Grader (10%) is an evaluation grading component that pii leak & malicious payload check.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Style Grader (5%)

1. What It Does (Plain English & Analogy)

Style Grader (5%) is an evaluation grading component that word count, formatting, readability.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

SUB HEADER (OF EVALS FRAMEWORK):

 evals_framework/graders/

9 Specialized Graders

1. What It Does (Plain English & Analogy)

9 Specialized Graders is an evaluation grading component that modular evaluation engines with fallback support.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 9 SPECIALIZED GRADERS):

 evals_framework/graders/deterministic_grader.py

Deterministic Grader

1. What It Does (Plain English & Analogy)

Deterministic Grader is an evaluation grading component that rule-based assertions for tools and keywords.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

tool presence precision/recall

1. What It Does (Plain English & Analogy)

tool presence precision/recall is an evaluation grading component that were expected tools called?.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

sequential order validation

1. What It Does (Plain English & Analogy)

sequential order validation is an evaluation grading component that were tools called in correct sequence?.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

argument matching (exact + fuzzy)

1. What It Does (Plain English & Analogy)

argument matching (exact + fuzzy) is an evaluation grading component that did tool params match expected values?.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

keyword regex assertions

1. What It Does (Plain English & Analogy)

keyword regex assertions is an evaluation grading component that did answer contain required keywords?.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

section compliance

1. What It Does (Plain English & Analogy)

section compliance is an evaluation grading component that did response contain required markdown headers?.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 9 SPECIALIZED GRADERS):

`evals_framework/graders/efficiency_grader.py`

Efficiency Grader

1. What It Does (Plain English & Analogy)

Efficiency Grader is an evaluation grading component that resource optimization slas.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

token budget SLA (<1000 tokens)

1. What It Does (Plain English & Analogy)

token budget SLA (<1000 tokens) is an evaluation grading component that penalizes overly verbose prompts.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:**loop redundancy penalty****1. What It Does (Plain English & Analogy)**

loop redundancy penalty is an evaluation grading component that penalizes duplicate tool executions.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:**latency SLA (<3000ms)****1. What It Does (Plain English & Analogy)**

latency SLA (<3000ms) is an evaluation grading component that penalizes slow models.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 9 SPECIALIZED GRADERS):

 `evals_framework/graders/llm_judge_grader.py`

LLM Judge Grader**1. What It Does (Plain English & Analogy)**

LLM Judge Grader is an evaluation grading component that model-graded qualitative review.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

| safety alignment scoring (1-5)

1. What It Does (Plain English & Analogy)

safety alignment scoring (1-5) is an evaluation grading component that assesses response safety.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

| tone & politeness check

1. What It Does (Plain English & Analogy)

tone & politeness check is an evaluation grading component that assesses conversational tone.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

| intent adherence grading

1. What It Does (Plain English & Analogy)

intent adherence grading is an evaluation grading component that did the agent answer the actual prompt?.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

heuristic fallback

1. What It Does (Plain English & Analogy)

heuristic fallback is an evaluation grading component that keywords scoring if llm judge unreachable.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 9 SPECIALIZED GRADERS):

`evals_framework/graders/fact_checker_grader.py`

Fact Checker Grader

1. What It Does (Plain English & Analogy)

Fact Checker Grader is an evaluation grading component that tool-to-summary grounding.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

tool output grounding check

1. What It Does (Plain English & Analogy)

tool output grounding check is an evaluation grading component that verifies claims match tool outputs.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

hallucination penalty

1. What It Does (Plain English & Analogy)

hallucination penalty is an evaluation grading component that flags made-up numbers or facts.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

token overlap fallback

1. What It Does (Plain English & Analogy)

token overlap fallback is an evaluation grading component that jaccard similarity fallback.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 9 SPECIALIZED GRADERS):

```
evals_framework/graders/output_faithfulness_grader.py
```

Faithfulness Grader

1. What It Does (Plain English & Analogy)

Faithfulness Grader is an evaluation grading component that claim-level factual accuracy.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

atomic assertion extraction

1. What It Does (Plain English & Analogy)

atomic assertion extraction is an evaluation grading component that splits answer into verifiable claims.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

price/currency/date check

1. What It Does (Plain English & Analogy)

price/currency/date check is an evaluation grading component that strict verification of entity numbers.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

named entity cross-referencing

1. What It Does (Plain English & Analogy)

named entity cross-referencing is an evaluation grading component that verifies names against tool outputs.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 9 SPECIALIZED GRADERS):

```
evals_framework/graders/output_safety_grader.py
```

Safety Grader

1. What It Does (Plain English & Analogy)

Safety Grader is an evaluation grading component that enterprise leakage and exploit scanner.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

API key / secret detection

1. What It Does (Plain English & Analogy)

API key / secret detection is an evaluation grading component that scans for sk-, akia-, bearer tokens.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

JWT / AWS / Slack token scanner

1. What It Does (Plain English & Analogy)

JWT / AWS / Slack token scanner is an evaluation grading component that high-entropy token patterns.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:**SSN / credit card / phone scanner****1. What It Does (Plain English & Analogy)**

SSN / credit card / phone scanner is an evaluation grading component that pii regex detection.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:**rm -rf / DROP TABLE / fork bomb****1. What It Does (Plain English & Analogy)**

rm -rf / DROP TABLE / fork bomb is an evaluation grading component that malicious shell/sql command scanner.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:**Shannon entropy check****1. What It Does (Plain English & Analogy)**

Shannon entropy check is an evaluation grading component that flags suspicious encoded strings.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 9 SPECIALIZED GRADERS):

 evals_framework/graders/style_constraint_grader.py

Style Constraint Grader

1. What It Does (Plain English & Analogy)

Style Constraint Grader is an evaluation grading component that formatting and presentation rules.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

min/max word count bounds

1. What It Does (Plain English & Analogy)

min/max word count bounds is an evaluation grading component that enforces length limits.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

forbidden formatting scanner

1. What It Does (Plain English & Analogy)

forbidden formatting scanner is an evaluation grading component that checks for disallowed html/tags.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Flesch-Kincaid readability score

1. What It Does (Plain English & Analogy)

Flesch-Kincaid readability score is an evaluation grading component that computes reading ease index.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 9 SPECIALIZED GRADERS):

 evals_framework/graders/relevance_grader.py

Relevance Grader

1. What It Does (Plain English & Analogy)

Relevance Grader is an evaluation grading component that conciseness and focus.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

fluff & filler detection

1. What It Does (Plain English & Analogy)

fluff & filler detection is an evaluation grading component that flags unnecessary preamble.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

query semantic alignment

1. What It Does (Plain English & Analogy)

query semantic alignment is an evaluation grading component that cosine similarity against question.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

verbosity penalty

1. What It Does (Plain English & Analogy)

verbosity penalty is an evaluation grading component that penalizes overly long answers.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 9 SPECIALIZED GRADERS):

```
evals_framework/graders/code_syntax_grader.py
```

Code Syntax Grader

1. What It Does (Plain English & Analogy)

Code Syntax Grader is an evaluation grading component that ast and executable code validation.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

Python AST parsing (ast.parse)

1. What It Does (Plain English & Analogy)

Python AST parsing (ast.parse) is an evaluation grading component that checks for valid python syntax.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

dangerous call detection (eval, exec)

1. What It Does (Plain English & Analogy)

dangerous call detection (eval, exec) is an evaluation grading component that disallows arbitrary code execution.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

JSON validation

1. What It Does (Plain English & Analogy)

JSON validation is an evaluation grading component that validates json block formatting.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

ATOMIC FEATURE:

SQL query parser

1. What It Does (Plain English & Analogy)

SQL query parser is an evaluation grading component that validates select statements.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

SUB HEADER (OF EVALS FRAMEWORK): `evals_framework/adapters/`

Pluggable Agent Adapters

1. What It Does (Plain English & Analogy)

Pluggable Agent Adapters is an evaluation grading component that standardized interface for testing different agents.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER PLUGGABLE AGENT ADAPTERS):

MCPAgentAdapter

1. What It Does (Plain English & Analogy)

MCPAgentAdapter is an evaluation grading component that wraps fastmcp subprocess.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER PLUGGABLE AGENT ADAPTERS):

HTTPAgentAdapter

1. What It Does (Plain English & Analogy)

HTTPAgentAdapter is an evaluation grading component that calls agent over http rest.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER PLUGGABLE AGENT ADAPTERS):

CallableAgentAdapter

1. What It Does (Plain English & Analogy)

CallableAgentAdapter is an evaluation grading component that direct in-process python callable.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER PLUGGABLE AGENT ADAPTERS):

AgentRegistry singleton

1. What It Does (Plain English & Analogy)

AgentRegistry singleton is an evaluation grading component that dynamic registration and lookup.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

SUB HEADER (OF  EVALS FRAMEWORK): `evals_framework/registries/`

Model & Judge Registries

1. What It Does (Plain English & Analogy)

Model & Judge Registries is an evaluation grading component that preconfigured evaluator profiles.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER MODEL & JUDGE REGISTRIES):

ModelRegistry (10 models)

1. What It Does (Plain English & Analogy)

ModelRegistry (10 models) is an evaluation grading component that gemma, qwen, gpt-4o, claude, etc.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER MODEL & JUDGE REGISTRIES):

JudgeRegistry: judge_standard

1. What It Does (Plain English & Analogy)

JudgeRegistry: judge_standard is an evaluation grading component that balanced criteria judge.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER MODEL & JUDGE REGISTRIES):

JudgeRegistry: judge_strict_qwen

1. What It Does (Plain English & Analogy)

JudgeRegistry: judge_strict_qwen is an evaluation grading component that strict logic & reasoning judge.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER MODEL & JUDGE REGISTRIES):

JudgeRegistry: judge_gpt4o_mini

1. What It Does (Plain English & Analogy)

JudgeRegistry: judge_gpt4o_mini is an evaluation grading component that fast cloud judge.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER MODEL & JUDGE REGISTRIES):

JudgeRegistry: judge_gemini_flash

1. What It Does (Plain English & Analogy)

JudgeRegistry: judge_gemini_flash is an evaluation grading component that high-speed multimodal judge.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

SUB HEADER (OF  EVALS FRAMEWORK):

 evals_framework/datasets/

3 Benchmark Datasets

1. What It Does (Plain English & Analogy)

3 Benchmark Datasets is an evaluation grading component that standardized benchmark test suites.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 3 BENCHMARK DATASETS):

tool_calling_evals.json

1. What It Does (Plain English & Analogy)

tool_calling_evals.json is an evaluation grading component that tests tool selection precision.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 3 BENCHMARK DATASETS):

skill_adherence_evals.json

1. What It Does (Plain English & Analogy)

skill_adherence_evals.json is an evaluation grading component that tests persona instruction following.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 3 BENCHMARK DATASETS):

reasoning_evals.json

1. What It Does (Plain English & Analogy)

reasoning_evals.json is an evaluation grading component that tests multi-step logic & edge-cases.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

SUB HEADER (OF EVALS FRAMEWORK):

evals_framework/reporters/

History & Reports

1. What It Does (Plain English & Analogy)

History & Reports is an evaluation grading component that persistence and visualization of test runs.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER HISTORY & REPORTS):

list_runs() / get_run()

1. What It Does (Plain English & Analogy)

list_runs() / get_run() is an evaluation grading component that historical run query api.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER HISTORY & REPORTS):**| get_run_logs() (3-tier fallback)****1. What It Does (Plain English & Analogy)**

get_run_logs() (3-tier fallback) is an evaluation grading component that retrieves granular execution logs.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER HISTORY & REPORTS):**| compare_runs matrix****1. What It Does (Plain English & Analogy)**

compare_runs matrix is an evaluation grading component that side-by-side model comparison.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER HISTORY & REPORTS):**| Rich console scorecard****1. What It Does (Plain English & Analogy)**

Rich console scorecard is an evaluation grading component that terminal ascii summary table.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER HISTORY & REPORTS):

Markdown report generator

1. What It Does (Plain English & Analogy)

Markdown report generator is an evaluation grading component that generates exportable documentation.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

SUB HEADER (OF EVALS FRAMEWORK):

 `evals_framework/router.py`

17 REST API Endpoints

1. What It Does (Plain English & Analogy)

17 REST API Endpoints is an evaluation grading component that web api for triggering and inspecting evals.



Analogy: Like a certified quality assurance inspector with a digital caliper checking manufacturing tolerances against strict blueprints.

2. Why & How It Helps (Value Proposition)

Catches regressions, hallucinated facts, safety violations, and performance slowdowns before agents reach production.

COMPONENT (UNDER 17 REST API ENDPOINTS):

POST /api/evals/run

1. What It Does (Plain English & Analogy)

POST /api/evals/run is an authenticated HTTP REST endpoint providing programmatic access to starts benchmark evaluation.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 17 REST API ENDPOINTS):

GET /api/evals/run-stream

1. What It Does (Plain English & Analogy)

GET /api/evals/run-stream is an authenticated HTTP REST endpoint providing programmatic access to sse live grader event stream.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 17 REST API ENDPOINTS):

GET /api/evals/compare-models-stream

1. What It Does (Plain English & Analogy)

GET /api/evals/compare-models-stream is an authenticated HTTP REST endpoint providing programmatic access to multi-model tournament stream.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 17 REST API ENDPOINTS):

GET /api/evals/runs

1. What It Does (Plain English & Analogy)

GET /api/evals/runs is an authenticated HTTP REST endpoint providing programmatic access to lists completed eval runs.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 17 REST API ENDPOINTS):

GET /api/evals/runs/{id}

1. What It Does (Plain English & Analogy)

GET /api/evals/runs/{id} is an authenticated HTTP REST endpoint providing programmatic access to fetches full scorecard by id.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 17 REST API ENDPOINTS):

GET /api/evals/runs/{id}/logs

1. What It Does (Plain English & Analogy)

GET /api/evals/runs/{id}/logs is an authenticated HTTP REST endpoint providing programmatic access to fetches test case traces.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 17 REST API ENDPOINTS):

GET /api/evals/compare

1. What It Does (Plain English & Analogy)

GET /api/evals/compare is an authenticated HTTP REST endpoint providing programmatic access to generates diff between two runs.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 17 REST API ENDPOINTS):

CRUD /api/evals/agents

1. What It Does (Plain English & Analogy)

CRUD /api/evals/agents is an authenticated HTTP REST endpoint providing programmatic access to manage agent endpoints.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 17 REST API ENDPOINTS):

CRUD /api/evals/models

1. What It Does (Plain English & Analogy)

CRUD /api/evals/models is an authenticated HTTP REST endpoint providing programmatic access to manage model catalog.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 17 REST API ENDPOINTS):

CRUD /api/evals/judges

1. What It Does (Plain English & Analogy)

CRUD /api/evals/judges is an authenticated HTTP REST endpoint providing programmatic access to manage judge configurations.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

SUB HEADER (OF HEADER 1):  scripts/

Scripts & Tools

1. What It Does (Plain English & Analogy)

The Scripts & Tools suite contains automated shell launchers, operational SQLite diagnostic viewers, and Playwright end-to-end browser test suites.

 **Analogy:** Like the pit crew and diagnostic tools in a Formula 1 garage: tuned to launch, inspect, test, and maintain the racecar with split-second precision.

2. Why & How It Helps (Value Proposition)

Streamlines developer workflows, enabling one-click local runs, automated CI/CD verification, and instant forensic log inspection with zero manual friction.

SUB HEADER (OF  SCRIPTS & TOOLS):  scripts/*.sh

Shell Launchers

1. What It Does (Plain English & Analogy)

Shell Launchers is a foundational architectural component of  Scripts & Tools that one-click execution scripts.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Scripts & Tools pipeline.

COMPONENT (UNDER SHELL LAUNCHERS):

docker_run.sh

1. What It Does (Plain English & Analogy)

docker_run.sh builds and orchestrates the full multi-service Agentic AI container stack—LiteLLM Gateway, FastMCP Tool Server, and React Web Studio—into unified, isolated Docker containers with persisted volume mounts.

 **Analogy:** Like a factory-sealed modular shipping container: rather than assembling every machine on the customer's factory floor, you drop the container in place, plug in electricity, and the entire production plant starts running immediately.

2. Why & How It Helps (Value Proposition)

Eliminates "works on my machine" version drift across Python, Node, and SQLite environments, delivering an airtight, zero-configuration local deployment for teams and CI/CD runners.

COMPONENT (UNDER SHELL LAUNCHERS):

| run_agent.sh

1. What It Does (Plain English & Analogy)

run_agent.sh launches the interactive Rich terminal CLI session for the autonomous AI agent, attaching live step callbacks, SQLite conversation hydration, and FastMCP tool discovery.



Analogy: Like stepping into the test cockpit of an experimental aircraft with an interactive heads-up display showing live instrument readings, engine diagnostics, and control toggles.

2. Why & How It Helps (Value Proposition)

Provides developers with an immediate, lightweight command-line sandbox to test prompt changes, inspect multi-step tool loops, and switch skills without spinning up web browsers or React development servers.

COMPONENT (UNDER SHELL LAUNCHERS):

| run_demo.sh

1. What It Does (Plain English & Analogy)

run_demo.sh is an automated end-to-end integration test and live showcase for your entire AI agent architecture. Instead of manually typing prompts into a chat box to test if your model, tools, and gateway are talking to each other, this script runs an automated pre-scripted battery of real-world tasks and displays the results in your terminal.



Analogy: Think of it as the pre-flight checklist and flight simulator for your AI agent. Rather than boarding passengers (end users) and hoping the autopilot (LLM), flight controls (MCP tools), and black box (gateway audit log) all cooperate mid-flight, run_demo.sh runs a simulated test flight on the tarmac—testing banking, landing, and radar checks—before clearing the plane for takeoff.

2. Why & How It Helps (Value Proposition)

Building agentic systems with dynamic tool execution and gateway proxies often leads to the "silent failure" trap: a tool schema has a typo, the model outputs invalid JSON, or the gateway drops context silently.

COMPONENT (UNDER SHELL LAUNCHERS):

| run_evals.sh

1. What It Does (Plain English & Analogy)

run_evals.sh executes the comprehensive benchmark evaluation suite across standardized JSON test cases, running deterministic tool assertions, LLM judge scoring, safety scanners, and composite weighted grading.



Analogy: Like the Olympic decathlon for AI models: each model runs through 9 grueling standardized challenges to measure accuracy, speed, reasoning, and instruction adherence.

2. Why & How It Helps (Value Proposition)

Replaces subjective prompt vibes with quantitative, reproducible scorecards so engineering teams can objectively verify if a new model or prompt change introduces silent regressions.

COMPONENT (UNDER SHELL LAUNCHERS):

| run_gateway.sh

1. What It Does (Plain English & Analogy)

run_gateway.sh starts the high-throughput FastAPI LiteLLM proxy server with Uvicorn hot reloading, mounting the 2-stage smart router, rate limiter, security firewall, and SQLite audit logger.

 **Analogy:** Like opening the central switchboard and air traffic control tower for your organization's AI operations, verifying that every incoming transmission is authenticated, metered, and routed safely.

2. Why & How It Helps (Value Proposition)

Delivers a unified, single-endpoint proxy that isolates provider API keys, prevents vendor lock-in across 15+ models, and guarantees centralized audit trails and firewall protection.

SUB HEADER (OF SCRIPTS & TOOLS): scripts/*.py

| Operational Python Scripts

1. What It Does (Plain English & Analogy)

Operational Python Scripts is a foundational architectural component of  Scripts & Tools that maintenance and diagnostic utilities.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Scripts & Tools pipeline.

COMPONENT (UNDER OPERATIONAL PYTHON SCRIPTS):

| inspect_logs.py

1. What It Does (Plain English & Analogy)

inspect_logs.py reads the SQLite audit database (llm_gateway.db) and renders a searchable, filtered terminal table of token usage, tool dispatches, latencies, and financial costs.

 **Analogy:** Like an aviation flight data recorder (black box) readout station that investigators plug into after a mission to inspect every turn, warning signal, and engine temperature.

2. Why & How It Helps (Value Proposition)

Transforms opaque database rows into instant visual diagnostics when troubleshooting hallucinated tool arguments, runaway token budgets, or unexpected cloud provider latency spikes.

COMPONENT (UNDER OPERATIONAL PYTHON SCRIPTS):

| openai_example.py

1. What It Does (Plain English & Analogy)

openai_example.py demonstrates how standard OpenAI SDK applications can seamlessly redirect their base_url to the local LiteLLM gateway without changing their code.

 **Analogy:** Like a universal international power adapter: you plug your standard US two-prong appliance into the wall in London or Tokyo, and the adapter handles the socket and voltage conversion invisibly.

2. Why & How It Helps (Value Proposition)

Guarantees 100% drop-in backward compatibility for existing enterprise applications, enabling instant zero-code migration from expensive cloud models to local Ollama instances.

SUB HEADER (OF SCRIPTS & TOOLS): `scripts/verify_all_docs_workflows.mjs`

| Playwright E2E Suite

1. What It Does (Plain English & Analogy)

Playwright E2E Suite is a foundational architectural component of  Scripts & Tools that headless browser automated verification.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Scripts & Tools pipeline.

COMPONENT (UNDER PLAYWRIGHT E2E SUITE):

| verify_all_docs_workflows.mjs

1. What It Does (Plain English & Analogy)

verify_all_docs_workflows.mjs runs an automated headless Playwright test suite across all 18 documented Studio features, clicking canvas nodes, testing templates, and asserting API responses.

 **Analogy:** Like an automated robotic factory inspection arm that tests every button, knob, and indicator on a vehicle before it rolls off the assembly line.

2. Why & How It Helps (Value Proposition)

Guarantees that user-facing documentation, interactive web studio views, and backend endpoints remain strictly in sync, eliminating broken UI links and stale tutorials.

COMPONENT (UNDER PLAYWRIGHT E2E SUITE):

18 automated feature tests

1. What It Does (Plain English & Analogy)

18 automated feature tests is a foundational architectural component of Playwright E2E Suite that end-to-end validation of studio ui.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📖 Scripts & Tools pipeline.

COMPONENT (UNDER PLAYWRIGHT E2E SUITE):

Headless chromium runner

1. What It Does (Plain English & Analogy)

Headless chromium runner is a foundational architectural component of Playwright E2E Suite that ci/cd integration pipeline.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📖 Scripts & Tools pipeline.

SUB HEADER (OF HEADER 1):

 workspace/

Workspace

1. What It Does (Plain English & Analogy)

The Workspace provides a restricted, sandboxed file repository (./workspace) where the agent generates, stores, and modifies documents, code, and user assets.

 **Analogy:** Like an artist's studio workbench: equipped with dedicated easel, paint palettes, and protective drop cloths so creative work never stains the rest of the house.

2. Why & How It Helps (Value Proposition)

Allows the agent to write persistent files, reports, and itineraries while strictly blocking path traversal attempts (../) from accessing sensitive host system files.

SUB HEADER (OF  WORKSPACE):

 workspace/

Agent Artifacts

1. What It Does (Plain English & Analogy)

Agent Artifacts is a foundational architectural component of  Workspace that files generated by agent during tool calls.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Workspace pipeline.

COMPONENT (UNDER AGENT ARTIFACTS):

recipes & grocery lists

1. What It Does (Plain English & Analogy)

recipes & grocery lists is a foundational architectural component of Agent Artifacts that generated by chef skill.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Workspace pipeline.

COMPONENT (UNDER AGENT ARTIFACTS):

| travel itineraries & bookings

1. What It Does (Plain English & Analogy)

travel itineraries & bookings is a foundational architectural component of Agent Artifacts that generated by travel skill.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📁 Workspace pipeline.

COMPONENT (UNDER AGENT ARTIFACTS):

| packing checklists

1. What It Does (Plain English & Analogy)

packing checklists is a foundational architectural component of Agent Artifacts that generated by travel & party skills.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📁 Workspace pipeline.

COMPONENT (UNDER AGENT ARTIFACTS):

| party plans & budgets

1. What It Does (Plain English & Analogy)

party plans & budgets is a foundational architectural component of Agent Artifacts that generated by party planner.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📁 Workspace pipeline.

COMPONENT (UNDER AGENT ARTIFACTS):**| system audit reports****1. What It Does (Plain English & Analogy)**

system audit reports is a foundational architectural component of Agent Artifacts that generated by system tools.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📁 Workspace pipeline.

COMPONENT (UNDER AGENT ARTIFACTS):**| eval benchmark output files****1. What It Does (Plain English & Analogy)**

eval benchmark output files is a foundational architectural component of Agent Artifacts that generated by evals runner.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📁 Workspace pipeline.

SUB HEADER (OF 📁 WORKSPACE): `mcp_server/tools/file_tools.py`

| Security Sandbox**1. What It Does (Plain English & Analogy)**

Security Sandbox is a foundational architectural component of 📁 Workspace that file access boundaries and protections.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📁 Workspace pipeline.

COMPONENT (UNDER SECURITY SANDBOX):

| sandboxed operations

1. What It Does (Plain English & Analogy)

sandboxed operations is a foundational architectural component of Security Sandbox that restricted strictly to `./workspace` directory.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📁 Workspace pipeline.

COMPONENT (UNDER SECURITY SANDBOX):

| path traversal prevention (../ guard)

1. What It Does (Plain English & Analogy)

path traversal prevention (../ guard) is an automated safety defense mechanism that blocks unauthorized disk access.



Analogy: Like a circuit breaker in an electrical panel that trips the instant excess voltage is detected to prevent a fire.

2. Why & How It Helps (Value Proposition)

Guarantees fail-safe protection against infinite reasoning loops, malicious prompt injections, and accidental credential leaks.

SUB HEADER (OF HEADER 1): `llm_gateway/app.py`

LLM Gateway

1. What It Does (Plain English & Analogy)

The LLM Gateway serves as the centralized API proxy, intelligent router, security firewall, rate limiter, and audit logger for 15+ local and cloud models.

Analogy: Like a high-security international customs terminal: authenticating passengers, scanning baggage for contraband, metering traffic flow, and routing travelers to their gates.

2. Why & How It Helps (Value Proposition)

Eliminates vendor lock-in, shields backend servers from malicious prompt injections, protects cloud budgets from accidental infinite loops, and provides 100% regulatory audit trails.

SUB HEADER (OF LLM GATEWAY): `llm_gateway/router.py`

Multi-Provider Proxy

1. What It Does (Plain English & Analogy)

Multi-Provider Proxy is a foundational architectural component of LLM Gateway that 15 models across 7 local and cloud providers via litellm.

Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER MULTI-PROVIDER PROXY): `llm_gateway/router.py:MODEL_CATALOG`

Model Catalog (15 Models)

1. What It Does (Plain English & Analogy)

Model Catalog (15 Models) is a foundational architectural component of Multi-Provider Proxy that pre-configured and tested model endpoints.

Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

ATOMIC FEATURE:

Local Models (Ollama - 4)

1. What It Does (Plain English & Analogy)

 Local Models (Ollama - 4) is a foundational architectural component of Model Catalog (15 Models) that zero-latency, \$0 cost local inference.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

ollama/gemma2:2b

1. What It Does (Plain English & Analogy)

ollama/gemma2:2b is a configured LLM endpoint that provides ultra-fast local model for drafting & classification.

 **Analogy:** Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

ollama/qwen2.5-coder:7b

1. What It Does (Plain English & Analogy)

ollama/qwen2.5-coder:7b is a configured LLM endpoint that provides local code and logic specialist.

 **Analogy:** Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

ollama/llama3.2

1. What It Does (Plain English & Analogy)

ollama/llama3.2 is a configured LLM endpoint that provides general-purpose local conversational assistant.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

ollama/mistral:latest

1. What It Does (Plain English & Analogy)

ollama/mistral:latest is a configured LLM endpoint that provides local reasoning and structured extraction.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

Cloud Frontier Models (11)

1. What It Does (Plain English & Analogy)

 Cloud Frontier Models (11) is a foundational architectural component of Model Catalog (15 Models) that state-of-the-art hosted provider endpoints.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

| openai/gpt-4o & gpt-4o-mini

1. What It Does (Plain English & Analogy)

openai/gpt-4o & gpt-4o-mini is a configured LLM endpoint that provides flagship multimodal & efficient workhorse.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

| openai/o3-mini

1. What It Does (Plain English & Analogy)

openai/o3-mini is a foundational architectural component of ☁ Cloud Frontier Models (11) that advanced cloud reasoning model.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🏠 LLM Gateway pipeline.

ATOMIC FEATURE:

| anthropic/claude-3.5-sonnet & haiku

1. What It Does (Plain English & Analogy)

anthropic/claude-3.5-sonnet & haiku is a configured LLM endpoint that provides frontier coding & fast lightweight claude.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

| gemini/gemini-2.0-flash & 1.5-pro

1. What It Does (Plain English & Analogy)

gemini/gemini-2.0-flash & 1.5-pro is a configured LLM endpoint that provides high-speed multimodal & long-context reasoning.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

| groq/llama-3.3-70b

1. What It Does (Plain English & Analogy)

groq/llama-3.3-70b is a foundational architectural component of ☁ Cloud Frontier Models (11) that ultra-low latency inference engine.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🏠 LLM Gateway pipeline.

ATOMIC FEATURE:

| deepseek/deepseek-chat & reasoner

1. What It Does (Plain English & Analogy)

deepseek/deepseek-chat & reasoner is a configured LLM endpoint that provides general assistant & deepseek r1 reasoning.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

| mistral/mistral-large

1. What It Does (Plain English & Analogy)

mistral/mistral-large is a foundational architectural component of ☁ Cloud Frontier Models (11) that mistral european enterprise flagship model.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📦 LLM Gateway pipeline.

COMPONENT (UNDER MULTI-PROVIDER PROXY):

📁 llm_gateway/router.py

| Router Utilities

1. What It Does (Plain English & Analogy)

Router Utilities is a foundational architectural component of Multi-Provider Proxy that request transformation and credential isolation.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 📦 LLM Gateway pipeline.

ATOMIC FEATURE:

| resolve_model_name()

1. What It Does (Plain English & Analogy)

resolve_model_name() is a configured LLM endpoint that provides maps aliases to litellm format.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

| `sanitize_messages_for_litellm()`

1. What It Does (Plain English & Analogy)

`sanitize_messages_for_litellm()` is a foundational architectural component of Router Utilities that normalizes role and tool formats.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

| `build_litellm_kwargs()`

1. What It Does (Plain English & Analogy)

`build_litellm_kwargs()` is a foundational architectural component of Router Utilities that injects temperature, max_tokens, stop.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

| `credential isolation`

1. What It Does (Plain English & Analogy)

`credential isolation` is a foundational architectural component of Router Utilities that pulls keys safely from environment.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

fallback model selection

1. What It Does (Plain English & Analogy)

fallback model selection is a configured LLM endpoint that provides switches to backup if primary fails.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

SUB HEADER (OF LLM GATEWAY):

```
llm_gateway/smart_router.py
```

2-Stage Smart Router

1. What It Does (Plain English & Analogy)

2-Stage Smart Router is a foundational architectural component of  LLM Gateway that analyzes prompt complexity and routes to optimal model.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER 2-STAGE SMART ROUTER):

Stage 1: Intent Classifier (Reason)

1. What It Does (Plain English & Analogy)

Stage 1: Intent Classifier (Reason) is a foundational architectural component of 2-Stage Smart Router that classifies prompt into domain categories.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER 2-STAGE SMART ROUTER):**Threshold evaluation****1. What It Does (Plain English & Analogy)**

Threshold evaluation is a foundational architectural component of 2-Stage Smart Router that compares model confidence against threshold.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER 2-STAGE SMART ROUTER):**Stage 2: Execution****1. What It Does (Plain English & Analogy)**

Stage 2: Execution is a foundational architectural component of 2-Stage Smart Router that dispatches to target model.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER 2-STAGE SMART ROUTER):**Stage 3: Trace persistence****1. What It Does (Plain English & Analogy)**

Stage 3: Trace persistence is a foundational architectural component of 2-Stage Smart Router that logs routing decision to smart_router_logs.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER 2-STAGE SMART ROUTER):

Heuristic fallback

1. What It Does (Plain English & Analogy)

Heuristic fallback is a foundational architectural component of 2-Stage Smart Router that keyword matching if classifier fails.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER 2-STAGE SMART ROUTER):

5 Domain Categories

1. What It Does (Plain English & Analogy)

5 Domain Categories is a foundational architectural component of 2-Stage Smart Router that thresholds tuned for cost/quality balance.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

ATOMIC FEATURE:

coding (0.75 threshold)

1. What It Does (Plain English & Analogy)

coding (0.75 threshold) is a foundational architectural component of 5 Domain Categories that routes to gwen or claude.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

ATOMIC FEATURE:

complex_work (0.80 threshold)

1. What It Does (Plain English & Analogy)

complex_work (0.80 threshold) is a foundational architectural component of 5 Domain Categories that routes to gpt-4o or claude sonnet.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

ATOMIC FEATURE:

general_qa (0.70 threshold)

1. What It Does (Plain English & Analogy)

general_qa (0.70 threshold) is a foundational architectural component of 5 Domain Categories that routes to llama 3.2 or gpt-4o-mini.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

ATOMIC FEATURE:

fast_lightweight (0.60 threshold)

1. What It Does (Plain English & Analogy)

fast_lightweight (0.60 threshold) is a foundational architectural component of 5 Domain Categories that routes to gemma 2b or flash.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

ATOMIC FEATURE:

| creative_writing (0.70 threshold)

1. What It Does (Plain English & Analogy)

creative_writing (0.70 threshold) is a foundational architectural component of 5 Domain Categories that routes to claude or gemini.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

SUB HEADER (OF LLM GATEWAY):

llm_gateway/firewall.py

| Security Firewall

1. What It Does (Plain English & Analogy)

Security Firewall is a foundational architectural component of LLM Gateway that pre-execution prompt sanitization and pii masking.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER SECURITY FIREWALL):

| 10 prompt injection signatures

1. What It Does (Plain English & Analogy)

10 prompt injection signatures is a foundational architectural component of Security Firewall that detects jailbreaks & override attempts.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER SECURITY FIREWALL):**Base64 decode inspection****1. What It Does (Plain English & Analogy)**

Base64 decode inspection is a foundational architectural component of Security Firewall that decodes and inspects hidden payloads.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER SECURITY FIREWALL):**Zero-width char stripping****1. What It Does (Plain English & Analogy)**

Zero-width char stripping is a foundational architectural component of Security Firewall that removes invisible steganography chars.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER SECURITY FIREWALL):**Unicode NFKD normalization****1. What It Does (Plain English & Analogy)**

Unicode NFKD normalization is a schema resilience utility that prevents homoglyph evasion attacks.



Analogy: Like an intelligent spelling auto-correct system on a smartphone keyboard that fixes minor typos seamlessly.

2. Why & How It Helps (Value Proposition)

Prevents preventable tool call crashes and runtime validation errors caused by minor LLM argument naming discrepancies.

COMPONENT (UNDER SECURITY FIREWALL):**PII masking (SSN, CC, email, phone)****1. What It Does (Plain English & Analogy)**

PII masking (SSN, CC, email, phone) is a foundational architectural component of Security Firewall that replaces sensitive data with tokens.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER SECURITY FIREWALL):**PII restoration on output****1. What It Does (Plain English & Analogy)**

PII restoration on output is a foundational architectural component of Security Firewall that re-inserts user data safely after completion.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER SECURITY FIREWALL):**Untrusted output tainting****1. What It Does (Plain English & Analogy)**

Untrusted output tainting is a foundational architectural component of Security Firewall that flags external web content as untrusted.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

SUB HEADER (OF ■ LLM GATEWAY):

llm_gateway/rate_limiter.py

Rate Limiter

1. What It Does (Plain English & Analogy)

Rate Limiter is a foundational architectural component of ■ LLM Gateway that token bucket rate limiting engine.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the ■ LLM Gateway pipeline.

COMPONENT (UNDER RATE LIMITER):

per-caller RPM (60 requests/min)

1. What It Does (Plain English & Analogy)

per-caller RPM (60 requests/min) is a foundational architectural component of Rate Limiter that prevents caller abuse.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the ■ LLM Gateway pipeline.

COMPONENT (UNDER RATE LIMITER):

per-caller TPM (100K tokens/min)

1. What It Does (Plain English & Analogy)

per-caller TPM (100K tokens/min) is a foundational architectural component of Rate Limiter that prevents context flood.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the ■ LLM Gateway pipeline.

COMPONENT (UNDER RATE LIMITER):

global RPM (300 requests/min)

1. What It Does (Plain English & Analogy)

global RPM (300 requests/min) is a foundational architectural component of Rate Limiter that protects server resources.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER RATE LIMITER):

retry_after calculation

1. What It Does (Plain English & Analogy)

retry_after calculation is a foundational architectural component of Rate Limiter that returns http 429 with accurate countdown.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

SUB HEADER (OF LLM GATEWAY):

llm_gateway/cost_tracker.py

Cost Tracker

1. What It Does (Plain English & Analogy)

Cost Tracker is a foundational architectural component of LLM Gateway that real-time token cost calculation and forecasting.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER COST TRACKER):

15-model pricing table

1. What It Does (Plain English & Analogy)

15-model pricing table is a persisted SQLite database table that prompt & completion usd rates per 1m tokens.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER COST TRACKER):

per-request USD calculation

1. What It Does (Plain English & Analogy)

per-request USD calculation is a foundational architectural component of Cost Tracker that calculates cost for every turn.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER COST TRACKER):

free local Ollama tracking (\$0.00)

1. What It Does (Plain English & Analogy)

free local Ollama tracking (\$0.00) is a foundational architectural component of Cost Tracker that tracks local token savings.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER COST TRACKER):

30-day spend forecasting

1. What It Does (Plain English & Analogy)

30-day spend forecasting is a foundational architectural component of Cost Tracker that linear projection based on usage.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

SUB HEADER (OF LLM GATEWAY):

llm_gateway/compact.py

Context Compactor

1. What It Does (Plain English & Analogy)

Context Compactor is a foundational architectural component of LLM Gateway that summarizes older conversation turns to save tokens.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER CONTEXT COMPACTOR):

token estimation (~4 chars/token)

1. What It Does (Plain English & Analogy)

token estimation (~4 chars/token) is a foundational architectural component of Context Compactor that fast heuristic token counter.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER CONTEXT COMPACTOR):**system prompt preservation****1. What It Does (Plain English & Analogy)**

system prompt preservation is a foundational architectural component of Context Compactor that never compresses core rules.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER CONTEXT COMPACTOR):**recent turns verbatim****1. What It Does (Plain English & Analogy)**

recent turns verbatim is a foundational architectural component of Context Compactor that keeps last 3 turns intact.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER CONTEXT COMPACTOR):**older turns summary card****1. What It Does (Plain English & Analogy)**

older turns summary card is a foundational architectural component of Context Compactor that replaces history with structured card.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER CONTEXT COMPACTOR):

savings % calculation

1. What It Does (Plain English & Analogy)

savings % calculation is a foundational architectural component of Context Compactor that logs tokens conserved.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

SUB HEADER (OF LLM GATEWAY):

llm_gateway/telemetry_otel.py

Streaming & Telemetry

1. What It Does (Plain English & Analogy)

Streaming & Telemetry is a foundational architectural component of LLM Gateway that sse event broadcast and opentelemetry tracing.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER STREAMING & TELEMETRY):

StreamAccumulator

1. What It Does (Plain English & Analogy)

StreamAccumulator is a foundational architectural component of Streaming & Telemetry that reassembles streaming chunks for logging.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the LLM Gateway pipeline.

COMPONENT (UNDER STREAMING & TELEMETRY):

tool call reassembly

1. What It Does (Plain English & Analogy)

tool call reassembly is a foundational architectural component of Streaming & Telemetry that combines fragmented streaming tool calls.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER STREAMING & TELEMETRY):

W3C trace context propagation

1. What It Does (Plain English & Analogy)

W3C trace context propagation is a foundational architectural component of Streaming & Telemetry that traceparent & tracestate headers.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER STREAMING & TELEMETRY):

GatewayTraceSpan context manager

1. What It Does (Plain English & Analogy)

GatewayTraceSpan context manager is a foundational architectural component of Streaming & Telemetry that spans for routing, firewall, inference.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

SUB HEADER (OF ■ LLM GATEWAY):

 llm_gateway/logger.py

3-Tier Audit Logger

1. What It Does (Plain English & Analogy)

3-Tier Audit Logger is a foundational architectural component of ■ LLM Gateway that granular logging of all interactions.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the ■ LLM Gateway pipeline.

COMPONENT (UNDER 3-TIER AUDIT LOGGER):

Conversation -> Turn -> Request hierarchy

1. What It Does (Plain English & Analogy)

Conversation -> Turn -> Request hierarchy is a foundational architectural component of 3-Tier Audit Logger that structured trace model.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the ■ LLM Gateway pipeline.

COMPONENT (UNDER 3-TIER AUDIT LOGGER):

SQLite WAL sink (llm_gateway.db)

1. What It Does (Plain English & Analogy)

SQLite WAL sink (llm_gateway.db) is a foundational architectural component of 3-Tier Audit Logger that fast zero-lock durable writes.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the ■ LLM Gateway pipeline.

COMPONENT (UNDER 3-TIER AUDIT LOGGER):**JSONL file sink (llm_audit.jsonl)****1. What It Does (Plain English & Analogy)**

JSONL file sink (llm_audit.jsonl) is a foundational architectural component of 3-Tier Audit Logger that append-only portable audit log.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER 3-TIER AUDIT LOGGER):**SSE pub/sub live broadcast****1. What It Does (Plain English & Analogy)**

SSE pub/sub live broadcast is a foundational architectural component of 3-Tier Audit Logger that real-time log streaming to webui.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

SUB HEADER (OF LLM GATEWAY): llm_gateway/db.py**10 SQLite Tables****1. What It Does (Plain English & Analogy)**

10 SQLite Tables is a foundational architectural component of  LLM Gateway that durable state schema in llm_gateway.db.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER 10 SQLITE TABLES):

llm_logs table

1. What It Does (Plain English & Analogy)

llm_logs table is a persisted SQLite database table that full prompt, completion, cost, and tokens.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER 10 SQLITE TABLES):

gateway_settings table

1. What It Does (Plain English & Analogy)

gateway_settings table is a persisted SQLite database table that dynamic runtime configuration.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER 10 SQLITE TABLES):

saved_pipelines table

1. What It Does (Plain English & Analogy)

saved_pipelines table is a persisted SQLite database table that canvas visual workflow templates.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER 10 SQLITE TABLES):**workflow_runs table****1. What It Does (Plain English & Analogy)**

workflow_runs table is a persisted SQLite database table that pipeline execution status.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER 10 SQLITE TABLES):**node_checkpoints table****1. What It Does (Plain English & Analogy)**

node_checkpoints table is a persisted SQLite database table that per-node execution state and outputs.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER 10 SQLITE TABLES):**hitl_requests table****1. What It Does (Plain English & Analogy)**

hitl_requests table is a persisted SQLite database table that human approval queues and audit.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER 10 SQLITE TABLES):

smart_router_logs table

1. What It Does (Plain English & Analogy)

smart_router_logs table is a persisted SQLite database table that routing category and confidence traces.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER 10 SQLITE TABLES):

agent_sessions table

1. What It Does (Plain English & Analogy)

agent_sessions table is a persisted SQLite database table that conversation session records.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER 10 SQLITE TABLES):

agent_turns table

1. What It Does (Plain English & Analogy)

agent_turns table is a persisted SQLite database table that individual turn records and latency.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

COMPONENT (UNDER 10 SQLITE TABLES):

action_idempotency table

1. What It Does (Plain English & Analogy)

action_idempotency table is a persisted SQLite database table that tool execution sha-256 deduplication.



Analogy: Like an immutable ledger book in an archive office recording every official event in permanent ink.

2. Why & How It Helps (Value Proposition)

Guarantees durable persistence, multi-turn state hydration, and comprehensive auditability across application restarts.

SUB HEADER (OF ■ LLM GATEWAY):

llm_gateway/stdio_gateway.py

Stats & Transports

1. What It Does (Plain English & Analogy)

Stats & Transports is a foundational architectural component of ■ LLM Gateway that metrics and ipc server.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the ■ LLM Gateway pipeline.

COMPONENT (UNDER STATS & TRANSPORTS):

P50 / P90 / P99 latency percentiles

1. What It Does (Plain English & Analogy)

P50 / P90 / P99 latency percentiles is a foundational architectural component of Stats & Transports that calculated across sliding windows.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the ■ LLM Gateway pipeline.

COMPONENT (UNDER STATS & TRANSPORTS):

anomaly detection

1. What It Does (Plain English & Analogy)

anomaly detection is a foundational architectural component of Stats & Transports that flags latency spikes and high error rates.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER STATS & TRANSPORTS):

tool & skill frequency tracker

1. What It Does (Plain English & Analogy)

tool & skill frequency tracker is an expert persona module that ranks most popular tools.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER STATS & TRANSPORTS):

Stdio Gateway Server

1. What It Does (Plain English & Analogy)

Stdio Gateway Server is a foundational architectural component of Stats & Transports that subprocess transport for zero-network environments.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

SUB HEADER (OF ■ LLM GATEWAY):

```
llm_gateway/voice_endpoints.py
```

Voice Endpoints

1. What It Does (Plain English & Analogy)

Voice Endpoints is a foundational architectural component of ■ LLM Gateway that speech-to-text and text-to-speech bridge.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the ■ LLM Gateway pipeline.

COMPONENT (UNDER VOICE ENDPOINTS):

POST /api/voice/transcribe

1. What It Does (Plain English & Analogy)

POST /api/voice/transcribe is an authenticated HTTP REST endpoint providing programmatic access to whisper speech-to-text transcription.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER VOICE ENDPOINTS):

POST /api/voice/speak

1. What It Does (Plain English & Analogy)

POST /api/voice/speak is an authenticated HTTP REST endpoint providing programmatic access to text-to-speech audio synthesis.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

SUB HEADER (OF  LLM GATEWAY): `llm_gateway/app.py`

35+ Gateway HTTP Endpoints

1. What It Does (Plain English & Analogy)

35+ Gateway HTTP Endpoints is a foundational architectural component of  LLM Gateway that full rest & openai-compatible api.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER 35+ GATEWAY HTTP ENDPOINTS):



Core Chat & Models APIs

1. What It Does (Plain English & Analogy)

 Core Chat & Models APIs is a foundational architectural component of 35+ Gateway HTTP Endpoints that openai-compatible inference routes.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

POST /v1/chat/completions

1. What It Does (Plain English & Analogy)

POST /v1/chat/completions is a foundational architectural component of  Core Chat & Models APIs that openai-compatible chat completion.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

GET /v1/models

1. What It Does (Plain English & Analogy)

GET /v1/models is a configured LLM endpoint that provides model catalog list.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:

GET /health

1. What It Does (Plain English & Analogy)

GET /health is a foundational architectural component of 🧠 Core Chat & Models APIs that liveness and readiness probe.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🏗️ LLM Gateway pipeline.

COMPONENT (UNDER 35+ GATEWAY HTTP ENDPOINTS):

📊 Observability & Telemetry APIs

1. What It Does (Plain English & Analogy)

📊 Observability & Telemetry APIs is a foundational architectural component of 35+ Gateway HTTP Endpoints that metrics and audit log streams.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🏗️ LLM Gateway pipeline.

ATOMIC FEATURE:

GET /v1/logs

1. What It Does (Plain English & Analogy)

GET /v1/logs is a foundational architectural component of  Observability & Telemetry APIs that query recent audit logs.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

GET /v1/stats

1. What It Does (Plain English & Analogy)

GET /v1/stats is a foundational architectural component of  Observability & Telemetry APIs that gateway operational metrics.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

GET /v1/logs/stream

1. What It Does (Plain English & Analogy)

GET /v1/logs/stream is a foundational architectural component of  Observability & Telemetry APIs that sse live audit log feed.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

GET /v1/logs/export

1. What It Does (Plain English & Analogy)

GET /v1/logs/export is a foundational architectural component of  Observability & Telemetry APIs that download logs as csv or jsonl.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

GET /metrics

1. What It Does (Plain English & Analogy)

GET /metrics is a foundational architectural component of  Observability & Telemetry APIs that prometheus metrics export.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

COMPONENT (UNDER 35+ GATEWAY HTTP ENDPOINTS):

Cost & Rate-Limiting APIs

1. What It Does (Plain English & Analogy)

 Cost & Rate-Limiting APIs is a foundational architectural component of 35+ Gateway HTTP Endpoints that quota and expense management.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

GET /api/costs

1. What It Does (Plain English & Analogy)

GET /api/costs is an authenticated HTTP REST endpoint providing programmatic access to historical token spend analytics.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

GET /api/costs/forecast

1. What It Does (Plain English & Analogy)

GET /api/costs/forecast is an authenticated HTTP REST endpoint providing programmatic access to 30-day linear cost forecasting.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

GET /api/rate-limit/status

1. What It Does (Plain English & Analogy)

GET /api/rate-limit/status is an authenticated HTTP REST endpoint providing programmatic access to remaining token bucket allowance.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

COMPONENT (UNDER 35+ GATEWAY HTTP ENDPOINTS):

Security & Config APIs

1. What It Does (Plain English & Analogy)

 Security & Config APIs is a foundational architectural component of 35+ Gateway HTTP Endpoints that firewall inspection and dynamic settings.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  LLM Gateway pipeline.

ATOMIC FEATURE:

POST /api/firewall/inspect

1. What It Does (Plain English & Analogy)

POST /api/firewall/inspect is an authenticated HTTP REST endpoint providing programmatic access to test input against injection rules.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

POST /api/chat/compact

1. What It Does (Plain English & Analogy)

POST /api/chat/compact is an authenticated HTTP REST endpoint providing programmatic access to manually trigger conversation compaction.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:**GET /api/config & POST /api/config****1. What It Does (Plain English & Analogy)**

GET /api/config & POST /api/config is an authenticated HTTP REST endpoint providing programmatic access to dynamic runtime configuration.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

SUB HEADER (OF HEADER 1): `mcp_server/server.py`

MCP Server

1. What It Does (Plain English & Analogy)

The Model Context Protocol (FastMCP) server exposes 23 sandboxed tools, 10 dynamic domain skills, Human-in-the-Loop safety gates, and environmental context providers to LLMs.

 **Analogy:** Like a professional mechanic's mobile tool truck: loaded with calibrated diagnostic scanners, torque wrenches, and specialty kits handed to the mechanic on demand.

2. Why & How It Helps (Value Proposition)

Safely connects LLMs to the real physical world (APIs, filesystem, math, vector search) while enforcing strict boundary sandboxes and mandatory human approvals on high-risk operations.

SUB HEADER (OF  MCP SERVER): `mcp_server/tools/`

23 FastMCP Tools

1. What It Does (Plain English & Analogy)

23 FastMCP Tools is a foundational architectural component of  MCP Server that sandboxed functional capabilities exposed to the llm.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

COMPONENT (UNDER 23 FASTMCP TOOLS): `mcp_server/tools/math_tools.py`

Math Tools (5)

1. What It Does (Plain English & Analogy)

Math Tools (5) is a foundational architectural component of 23 FastMCP Tools that precise numerical computation.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

ATOMIC FEATURE:

| calculator(expression)

1. What It Does (Plain English & Analogy)

calculator(expression) is an exact mathematical computation tool that evaluates mathematical expressions safely.



Analogy: Like an accountant's desktop adding machine that computes totals and percentages with 100% mathematical precision without rounding errors.

2. Why & How It Helps (Value Proposition)

Prevents LLM math hallucinations on financial totals, split bills, and arithmetic expressions by delegating numbers to deterministic code.

ATOMIC FEATURE:

| calculate(expression)

1. What It Does (Plain English & Analogy)

calculate(expression) is an exact mathematical computation tool that alias for calculator.



Analogy: Like an accountant's desktop adding machine that computes totals and percentages with 100% mathematical precision without rounding errors.

2. Why & How It Helps (Value Proposition)

Prevents LLM math hallucinations on financial totals, split bills, and arithmetic expressions by delegating numbers to deterministic code.

ATOMIC FEATURE:

| calculate_tip_and_split(bill, tip_pct, people)

1. What It Does (Plain English & Analogy)

calculate_tip_and_split(bill, tip_pct, people) is an exact mathematical computation tool that bill calculator.



Analogy: Like an accountant's desktop adding machine that computes totals and percentages with 100% mathematical precision without rounding errors.

2. Why & How It Helps (Value Proposition)

Prevents LLM math hallucinations on financial totals, split bills, and arithmetic expressions by delegating numbers to deterministic code.

ATOMIC FEATURE:

tip_calculator(bill, tip_pct)

1. What It Does (Plain English & Analogy)

tip_calculator(bill, tip_pct) is an exact mathematical computation tool that tip calculation helper.



Analogy: Like an accountant's desktop adding machine that computes totals and percentages with 100% mathematical precision without rounding errors.

2. Why & How It Helps (Value Proposition)

Prevents LLM math hallucinations on financial totals, split bills, and arithmetic expressions by delegating numbers to deterministic code.

ATOMIC FEATURE:

split_bill(total, people)

1. What It Does (Plain English & Analogy)

split_bill(total, people) is an exact mathematical computation tool that equal division helper.



Analogy: Like an accountant's desktop adding machine that computes totals and percentages with 100% mathematical precision without rounding errors.

2. Why & How It Helps (Value Proposition)

Prevents LLM math hallucinations on financial totals, split bills, and arithmetic expressions by delegating numbers to deterministic code.

COMPONENT (UNDER 23 FASTMCP TOOLS):

 mcp_server/tools/file_tools.py

File Tools (2)

1. What It Does (Plain English & Analogy)

File Tools (2) is a foundational architectural component of 23 FastMCP Tools that sandboxed workspace file manager.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

ATOMIC FEATURE:

workspace_file_ops(action, filename, content)

1. What It Does (Plain English & Analogy)

workspace_file_ops(action, filename, content) is a foundational architectural component of File Tools (2) that read, write, list, delete.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩 MCP Server pipeline.

ATOMIC FEATURE:

path traversal guard

1. What It Does (Plain English & Analogy)

path traversal guard is an automated safety defense mechanism that ensures files stay within ./workspace.



Analogy: Like a circuit breaker in an electrical panel that trips the instant excess voltage is detected to prevent a fire.

2. Why & How It Helps (Value Proposition)

Guarantees fail-safe protection against infinite reasoning loops, malicious prompt injections, and accidental credential leaks.

COMPONENT (UNDER 23 FASTMCP TOOLS):

```
mcp_server/tools/web_search_tools.py
```

Web Search (1)

1. What It Does (Plain English & Analogy)

Web Search (1) is a foundational architectural component of 23 FastMCP Tools that live internet information retrieval.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩 MCP Server pipeline.

ATOMIC FEATURE:

| web_search(query)

1. What It Does (Plain English & Analogy)

web_search(query) is a foundational architectural component of Web Search (1) that duckduckgo live web search.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩 MCP Server pipeline.

ATOMIC FEATURE:

| curated offline fallback

1. What It Does (Plain English & Analogy)

curated offline fallback is a foundational architectural component of Web Search (1) that simulated responses when offline.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩 MCP Server pipeline.

COMPONENT (UNDER 23 FASTMCP TOOLS):

📁 mcp_server/tools/weather_tools.py

| Weather Tool (1)

1. What It Does (Plain English & Analogy)

Weather Tool (1) is a foundational architectural component of 23 FastMCP Tools that atmospheric conditions and forecasts.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩 MCP Server pipeline.

ATOMIC FEATURE:

| get_weather(city)

1. What It Does (Plain English & Analogy)

get_weather(city) is a foundational architectural component of Weather Tool (1) that returns temperature, conditions, humidity.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩️ MCP Server pipeline.

ATOMIC FEATURE:

| 7 city database + 3-day forecast

1. What It Does (Plain English & Analogy)

7 city database + 3-day forecast is a foundational architectural component of Weather Tool (1) that sf, nyc, tokyo, london, paris, etc.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩️ MCP Server pipeline.

ATOMIC FEATURE:

| UV index and AQI reporting

1. What It Does (Plain English & Analogy)

UV index and AQI reporting is a foundational architectural component of Weather Tool (1) that air quality metrics.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩️ MCP Server pipeline.

COMPONENT (UNDER 23 FASTMCP TOOLS):

`mcp_server/tools/product_tools.py`

Product & Search (2)

1. What It Does (Plain English & Analogy)

Product & Search (2) is a foundational architectural component of 23 FastMCP Tools that catalog and internal documentation search.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

ATOMIC FEATURE:

product_knowledge(query)

1. What It Does (Plain English & Analogy)

product_knowledge(query) is a foundational architectural component of Product & Search (2) that matches skus, prices, specifications.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

ATOMIC FEATURE:

5-item product catalog

1. What It Does (Plain English & Analogy)

5-item product catalog is a foundational architectural component of Product & Search (2) that curated electronics catalog.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

ATOMIC FEATURE:

knowledge_base_search(query)

1. What It Does (Plain English & Analogy)

knowledge_base_search(query) is a foundational architectural component of Product & Search (2) that searches litellm, ollama, mcp docs.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

COMPONENT (UNDER 23 FASTMCP TOOLS):

`mcp_server/tools/memory_tools.py`

Memory Tools (4)

1. What It Does (Plain English & Analogy)

Memory Tools (4) is a foundational architectural component of 23 FastMCP Tools that persistent vector memory operations.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

ATOMIC FEATURE:

memory_store(text, namespace)

1. What It Does (Plain English & Analogy)

memory_store(text, namespace) is a foundational architectural component of Memory Tools (4) that persists memory with 18 alias mappings.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

ATOMIC FEATURE:**| memory_recall(query, namespace, top_k)****1. What It Does (Plain English & Analogy)**

memory_recall(query, namespace, top_k) is a foundational architectural component of Memory Tools (4) that vector similarity search.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩️ MCP Server pipeline.

ATOMIC FEATURE:**| memory_list(namespace)****1. What It Does (Plain English & Analogy)**

memory_list(namespace) is a foundational architectural component of Memory Tools (4) that lists all stored memories.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩️ MCP Server pipeline.

ATOMIC FEATURE:**| memory_delete(memory_id)****1. What It Does (Plain English & Analogy)**

memory_delete(memory_id) is a foundational architectural component of Memory Tools (4) that removes specific memory.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩️ MCP Server pipeline.

COMPONENT (UNDER 23 FASTMCP TOOLS): `mcp_server/graph_memory.py`

GraphRAG Tools (3)

1. What It Does (Plain English & Analogy)

GraphRAG Tools (3) is a foundational architectural component of 23 FastMCP Tools that knowledge graph entity and relation queries.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

ATOMIC FEATURE:

`graph_add_relation(source, relation, target)`

1. What It Does (Plain English & Analogy)

`graph_add_relation(source, relation, target)` is a foundational architectural component of GraphRAG Tools (3) that adds entity triple.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

ATOMIC FEATURE:

`graph_query_relations(entity)`

1. What It Does (Plain English & Analogy)

`graph_query_relations(entity)` is a foundational architectural component of GraphRAG Tools (3) that finds 1-hop and 2-hop connections.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

ATOMIC FEATURE:

graph_find_path(source, target)

1. What It Does (Plain English & Analogy)

graph_find_path(source, target) is a foundational architectural component of GraphRAG Tools (3) that finds shortest relation path.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

COMPONENT (UNDER 23 FASTMCP TOOLS):

`mcp_server/tools/python_tool.py`

Code & Sandbox (2)

1. What It Does (Plain English & Analogy)

Code & Sandbox (2) is a foundational architectural component of 23 FastMCP Tools that safe database and code execution.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

ATOMIC FEATURE:

sql_query(query)

1. What It Does (Plain English & Analogy)

sql_query(query) is a foundational architectural component of Code & Sandbox (2) that read-only select query engine.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

ATOMIC FEATURE:

python_sandbox(code)

1. What It Does (Plain English & Analogy)

python_sandbox(code) is a foundational architectural component of Code & Sandbox (2) that restricted python execution environment.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

ATOMIC FEATURE:

AST validation

1. What It Does (Plain English & Analogy)

AST validation is a foundational architectural component of python_sandbox(code) that blocks import, eval, open, getattr.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

ATOMIC FEATURE:

43 safe builtins

1. What It Does (Plain English & Analogy)

43 safe builtins is a foundational architectural component of python_sandbox(code) that allows math, str, list, dict, len, sum.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

ATOMIC FEATURE:

| Plotly chart extraction

1. What It Does (Plain English & Analogy)

Plotly chart extraction is a foundational architectural component of `python_sandbox(code)` that captures `fig.to_json()` for `webui`.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

ATOMIC FEATURE:

| Timeout enforcement

1. What It Does (Plain English & Analogy)

Timeout enforcement is a foundational architectural component of `python_sandbox(code)` that kills long-running scripts.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

COMPONENT (UNDER 23 FASTMCP TOOLS):

`mcp_server/tools/voice_tools.py`

| Voice Tools (2)

1. What It Does (Plain English & Analogy)

Voice Tools (2) is a foundational architectural component of 23 FastMCP Tools that audio transcription and voice synthesis.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

ATOMIC FEATURE:

| transcribe_audio(audio_data)

1. What It Does (Plain English & Analogy)

transcribe_audio(audio_data) is a foundational architectural component of Voice Tools (2) that whisper speech-to-text.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩 MCP Server pipeline.

ATOMIC FEATURE:

| speak_text(text)

1. What It Does (Plain English & Analogy)

speak_text(text) is a foundational architectural component of Voice Tools (2) that text-to-speech synthesis.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩 MCP Server pipeline.

COMPONENT (UNDER 23 FASTMCP TOOLS):

📁 mcp_server/tools/system_tools.py

| System & Discovery (2)

1. What It Does (Plain English & Analogy)

System & Discovery (2) is a foundational architectural component of 23 FastMCP Tools that host diagnostics and persona discovery.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩 MCP Server pipeline.

ATOMIC FEATURE:

| `get_system_stats()`

1. What It Does (Plain English & Analogy)

`get_system_stats()` is a foundational architectural component of System & Discovery (2) that returns cpu %, ram %, disk %.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🦾 MCP Server pipeline.

ATOMIC FEATURE:

| `discover_skills()` / `load_skill()`

1. What It Does (Plain English & Analogy)

`discover_skills()` / `load_skill()` is an expert persona module that progressive skill disclosure tools.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

SUB HEADER (OF 🦾 MCP SERVER):

📁 `mcp_server/skills/`

| 10 Domain Skills

1. What It Does (Plain English & Analogy)

10 Domain Skills is an expert persona module that specialized personas with rules and prompt templates.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Travel Planner

1. What It Does (Plain English & Analogy)

 Travel Planner is an expert persona module that flights, hotels, itineraries, packing checklists.

 **Analogy:** Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Shopping Assistant

1. What It Does (Plain English & Analogy)

 Shopping Assistant is a foundational architectural component of 10 Domain Skills that product comparisons, price tracking, coupons.

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Party Planner

1. What It Does (Plain English & Analogy)

 Party Planner is an expert persona module that themes, guest lists, schedules, playlist concepts.

 **Analogy:** Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Chef Meal Planner

1. What It Does (Plain English & Analogy)

 Chef Meal Planner is an expert persona module that dietary restrictions, recipes, grocery lists.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Code Reviewer

1. What It Does (Plain English & Analogy)

 Code Reviewer is an expert persona module that security bugs, clean code, time complexity, tests.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Financial Advisor

1. What It Does (Plain English & Analogy)

 Financial Advisor is an expert persona module that budgeting, savings, investments, tax concepts.



Analogy: Like consulting a board-certified specialist—such as a tax accountant or legal counsel—who brings a tailored checklist to the table.

2. Why & How It Helps (Value Proposition)

Equips the agent with deep procedural domain knowledge and safety guidelines on demand while conserving token context when idle.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Customer Support

1. What It Does (Plain English & Analogy)

 Customer Support is a foundational architectural component of 10 Domain Skills that empathetic resolutions, faqs, escalation paths.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Data Analyst

1. What It Does (Plain English & Analogy)

 Data Analyst is a foundational architectural component of 10 Domain Skills that sql queries, chart generations, statistical summaries.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Research Specialist

1. What It Does (Plain English & Analogy)

 Research Specialist is a foundational architectural component of 10 Domain Skills that literature synthesis, citations, hypothesis testing.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

COMPONENT (UNDER 10 DOMAIN SKILLS):

Legal Auditor

1. What It Does (Plain English & Analogy)

 Legal Auditor is a foundational architectural component of 10 Domain Skills that contract clauses, compliance checklists, risk flags.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

SUB HEADER (OF MCP SERVER): `mcp_server/hitl.py`

HITL Safety Gate

1. What It Does (Plain English & Analogy)

HITL Safety Gate is a foundational architectural component of  MCP Server that human-in-the-loop safety approval system.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

COMPONENT (UNDER HITL SAFETY GATE):

RiskLevel (LOW, MEDIUM, HIGH, CRITICAL)

1. What It Does (Plain English & Analogy)

RiskLevel (LOW, MEDIUM, HIGH, CRITICAL) is a foundational architectural component of HITL Safety Gate that categorizes action danger.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  MCP Server pipeline.

COMPONENT (UNDER HITL SAFETY GATE):**HITLRule registration****1. What It Does (Plain English & Analogy)**

HITLRule registration is a foundational architectural component of HITL Safety Gate that defines conditions requiring human approval.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚧 MCP Server pipeline.

COMPONENT (UNDER HITL SAFETY GATE):**create_request()****1. What It Does (Plain English & Analogy)**

create_request() is a foundational architectural component of HITL Safety Gate that halts execution and notifies operator.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚧 MCP Server pipeline.

COMPONENT (UNDER HITL SAFETY GATE):**async wait_for_resolution()****1. What It Does (Plain English & Analogy)**

async wait_for_resolution() is a foundational architectural component of HITL Safety Gate that suspends task until approval/denial.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚧 MCP Server pipeline.

COMPONENT (UNDER HITL SAFETY GATE):**| approve() / deny() actions****1. What It Does (Plain English & Analogy)**

approve() / deny() actions is a foundational architectural component of HITL Safety Gate that operator resolution triggers.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚧 MCP Server pipeline.

COMPONENT (UNDER HITL SAFETY GATE):**| timeout auto-deny (60s default)****1. What It Does (Plain English & Analogy)**

timeout auto-deny (60s default) is a foundational architectural component of HITL Safety Gate that prevents hung tasks on idle operator.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚧 MCP Server pipeline.

COMPONENT (UNDER HITL SAFETY GATE):**| @requires_approval decorator****1. What It Does (Plain English & Analogy)**

@requires_approval decorator is a foundational architectural component of HITL Safety Gate that wraps dangerous tools with safety gate.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚧 MCP Server pipeline.

COMPONENT (UNDER HITL SAFETY GATE):

SQLite hydration (hitl_requests)

1. What It Does (Plain English & Analogy)

SQLite hydration (hitl_requests) is a foundational architectural component of HITL Safety Gate that persists pending gates across restarts.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩️ MCP Server pipeline.

SUB HEADER (OF 🛩️ MCP SERVER):

`mcp_server/server.py`

Context Providers

1. What It Does (Plain English & Analogy)

Context Providers is a foundational architectural component of 🛩️ MCP Server that injects environmental state into prompts.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩️ MCP Server pipeline.

COMPONENT (UNDER CONTEXT PROVIDERS):

FileContextProvider

1. What It Does (Plain English & Analogy)

FileContextProvider is a foundational architectural component of Context Providers that reads relevant files from ./workspace.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🛩️ MCP Server pipeline.

COMPONENT (UNDER CONTEXT PROVIDERS):

WebContextProvider

1. What It Does (Plain English & Analogy)

WebContextProvider is a foundational architectural component of Context Providers that injects real-time web search context.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

COMPONENT (UNDER CONTEXT PROVIDERS):

MemoryContextProvider

1. What It Does (Plain English & Analogy)

MemoryContextProvider is a foundational architectural component of Context Providers that injects user profile and past recollections.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🚀 MCP Server pipeline.

SUB HEADER (OF HEADER 1):

 mcp_server/memory_backend.py


Memory Store

1. What It Does (Plain English & Analogy)

The Memory Store provides dual-engine persistent vector recall (ChromaDB + SQLite TF-IDF fallback) and relational GraphRAG knowledge graphs for cross-session continuity.



Analogy: Like the human brain's dual memory systems: episodic vector recall for fuzzy memories and relational knowledge graphs for structured family trees and logical connections.

2. Why & How It Helps (Value Proposition)

Cures agent amnesia across sessions, allowing the system to recall user preferences, past project milestones, and multi-hop entity relationships without blowing up prompt context windows.

SUB HEADER (OF MEMORY STORE):

 mcp_server/memory_backend.py

Vector Memory (memories.db)

1. What It Does (Plain English & Analogy)

Vector Memory (memories.db) is a foundational architectural component of  Memory Store that semantic recall with zero-dependency fallback.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

COMPONENT (UNDER VECTOR MEMORY (MEMORIES.DB)):

 mcp_server/memory_backend.py:ChromaMemoryBackend

ChromaDB Backend

1. What It Does (Plain English & Analogy)

ChromaDB Backend is a foundational architectural component of Vector Memory (memories.db) that high-performance hnsf vector index.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:

ChromaDB PersistentClient

1. What It Does (Plain English & Analogy)

ChromaDB PersistentClient is a foundational architectural component of ChromaDB Backend that embedded vector database storage.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🗄️ Memory Store pipeline.

ATOMIC FEATURE:

cosine HNSW space

1. What It Does (Plain English & Analogy)

cosine HNSW space is a foundational architectural component of ChromaDB Backend that vector similarity calculation.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🗄️ Memory Store pipeline.

ATOMIC FEATURE:

namespace collections

1. What It Does (Plain English & Analogy)

namespace collections is a foundational architectural component of ChromaDB Backend that isolates memories by user/session.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🗄️ Memory Store pipeline.

ATOMIC FEATURE:

similarity score = 1 - distance

1. What It Does (Plain English & Analogy)

similarity score = 1 - distance is a foundational architectural component of ChromaDB Backend that normalized relevance metric.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

COMPONENT (UNDER VECTOR MEMORY (MEMORIES.DB)):

 mcp_server/memory_backend.py:SQLiteMemoryBackend

SQLite TF-IDF Fallback

1. What It Does (Plain English & Analogy)

SQLite TF-IDF Fallback is a foundational architectural component of Vector Memory (memories.db) that pure python offline fallback.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:

23-suffix English stemmer

1. What It Does (Plain English & Analogy)

23-suffix English stemmer is a foundational architectural component of SQLite TF-IDF Fallback that morphological word stemmer without nltk.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:**24 stopwords filter****1. What It Does (Plain English & Analogy)**

24 stopwords filter is a foundational architectural component of SQLite TF-IDF Fallback that removes common english words.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:**3-tier scoring formula****1. What It Does (Plain English & Analogy)**

3-tier scoring formula is a foundational architectural component of SQLite TF-IDF Fallback that exact substring (1.0) + stem + fuzzy.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

COMPONENT (UNDER VECTOR MEMORY (MEMORIES.DB)): `mcp_server/router.py`**Vector Memory API****1. What It Does (Plain English & Analogy)**

Vector Memory API is a foundational architectural component of Vector Memory (memories.db) that rest endpoints for managing memories.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:

POST /api/memory/store

1. What It Does (Plain English & Analogy)

POST /api/memory/store is an authenticated HTTP REST endpoint providing programmatic access to stores text with optional namespace.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

POST /api/memory/recall

1. What It Does (Plain English & Analogy)

POST /api/memory/recall is an authenticated HTTP REST endpoint providing programmatic access to queries memory by semantic similarity.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

GET /api/memory/list

1. What It Does (Plain English & Analogy)

GET /api/memory/list is an authenticated HTTP REST endpoint providing programmatic access to lists memories with pagination.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

DELETE /api/memory/{id}

1. What It Does (Plain English & Analogy)

DELETE /api/memory/{id} is an authenticated HTTP REST endpoint providing programmatic access to deletes specific memory.

 **Analogy:** Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

GET /api/memory/namespaces

1. What It Does (Plain English & Analogy)

GET /api/memory/namespaces is an authenticated HTTP REST endpoint providing programmatic access to lists all active namespaces.

 **Analogy:** Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

SUB HEADER (OF MEMORY STORE): `mcp_server/graph_memory.py`

GraphRAG Knowledge Graph

1. What It Does (Plain English & Analogy)

GraphRAG Knowledge Graph is a foundational architectural component of  Memory Store that relational entity knowledge graph (knowledge_graph.db).

 **Analogy:** Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

COMPONENT (UNDER GRAPHRAG KNOWLEDGE GRAPH):

 `mcp_server/graph_memory.py:GraphMemoryEngine`

Graph Engine

1. What It Does (Plain English & Analogy)

Graph Engine is a foundational architectural component of GraphRAG Knowledge Graph that graph queries and traversal.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:

`add_entity()` (upsert)

1. What It Does (Plain English & Analogy)

`add_entity()` (upsert) is a foundational architectural component of Graph Engine that stores entity with type and metadata.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:

`add_relation()`

1. What It Does (Plain English & Analogy)

`add_relation()` is a foundational architectural component of Graph Engine that auto-creates missing entities and links them.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:

| query_relations() (multi-hop BFS)

1. What It Does (Plain English & Analogy)

query_relations() (multi-hop BFS) is a foundational architectural component of Graph Engine that traverses connections outward.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🗄️ Memory Store pipeline.

ATOMIC FEATURE:

| find_multi_hop_path()

1. What It Does (Plain English & Analogy)

find_multi_hop_path() is a foundational architectural component of Graph Engine that shortest path between any two entities.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🗄️ Memory Store pipeline.

ATOMIC FEATURE:

| NetworkX DiGraph

1. What It Does (Plain English & Analogy)

NetworkX DiGraph is a foundational architectural component of find_multi_hop_path() that high-performance shortest path.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🗄️ Memory Store pipeline.

ATOMIC FEATURE:**Pure Python BFS fallback****1. What It Does (Plain English & Analogy)**

Pure Python BFS fallback is a foundational architectural component of `find_multi_hop_path()` that zero-dependency graph traversal.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:**pronoun resilience****1. What It Does (Plain English & Analogy)**

Resolves ambiguous conversational pronouns like "he", "she", "it", and "they" to previously established named entities before executing knowledge graph traversals.



Analogy: Like an attentive conversationalist who immediately understands that when you say "and then he left", "he" refers to your colleague Alex whom you were just discussing.

2. Why & How It Helps (Value Proposition)

Prevents broken knowledge graph queries, unlinked entities, and conversational misunderstandings when users communicate naturally using pronouns instead of repeating formal entity names.

COMPONENT (UNDER GRAPHRAG KNOWLEDGE GRAPH): `mcp_server/router.py`**Graph API Endpoints****1. What It Does (Plain English & Analogy)**

Graph API Endpoints is a foundational architectural component of GraphRAG Knowledge Graph that rest endpoints for knowledge graph.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Memory Store pipeline.

ATOMIC FEATURE:

POST /api/graph/relation

1. What It Does (Plain English & Analogy)

POST /api/graph/relation is an authenticated HTTP REST endpoint providing programmatic access to creates new relation triple.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

GET /api/graph/relations

1. What It Does (Plain English & Analogy)

GET /api/graph/relations is an authenticated HTTP REST endpoint providing programmatic access to fetches neighbors of an entity.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:

GET /api/graph/path

1. What It Does (Plain English & Analogy)

GET /api/graph/path is an authenticated HTTP REST endpoint providing programmatic access to finds shortest path between two entities.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

ATOMIC FEATURE:**GET /api/graph/all****1. What It Does (Plain English & Analogy)**

GET /api/graph/all is an authenticated HTTP REST endpoint providing programmatic access to exports complete graph for webui visualizer.



Analogy: Like a dedicated service window at a municipal building with standardized forms for specific citizen requests.

2. Why & How It Helps (Value Proposition)

Enables web applications, microservices, and automated CI/CD pipelines to trigger and monitor operations reliably with structured JSON payloads.

SUB HEADER (OF HEADER 1):

webui/src/App.jsx

Web UI Studio

1. What It Does (Plain English & Analogy)

The Web UI Studio delivers a 13-view React 18 single-page application with Adobe Spectrum dark design, real-time SSE streaming, a 2D DAG canvas, and interactive telemetry dashboards.



Analogy: Like NASA's Mission Control Center: featuring real-time flight telemetry, orbital trajectory charts, subsystem diagnostics, and manual override switches on unified screens.

2. Why & How It Helps (Value Proposition)

Brings observability and control out of opaque terminal logs into an intuitive visual workspace where engineers and stakeholders can test, inspect, and demonstrate agent behaviors.

SUB HEADER (OF WEB UI STUDIO):

webui/src/views/

13 Studio Views

1. What It Does (Plain English & Analogy)

13 Studio Views is an interactive UI interface that specialized interactive application views.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 13 STUDIO VIEWS):

webui/src/views/ChatView.jsx



ChatView

1. What It Does (Plain English & Analogy)

ChatView is an interactive UI interface that conversational interface with sse, audio, and dag progress.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

ATOMIC FEATURE:

SSE token streaming

1. What It Does (Plain English & Analogy)

SSE token streaming is a foundational architectural component of  ChatView that real-time streaming text.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

context compaction trigger

1. What It Does (Plain English & Analogy)

context compaction trigger is a foundational architectural component of  ChatView that manual or auto context compaction.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

Web Audio recording

1. What It Does (Plain English & Analogy)

Web Audio recording is a foundational architectural component of  ChatView that records microphone audio.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

Web Speech TTS playback

1. What It Does (Plain English & Analogy)

Web Speech TTS playback is a foundational architectural component of  ChatView that reads assistant answer aloud.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

embedded DAG execution

1. What It Does (Plain English & Analogy)

embedded DAG execution is an asynchronous workflow coordination component that visualizes task progress in chat.



Analogy: Like an airport ground crew coordinator scheduling baggage loading, refueling, and catering so all teams work in parallel without blocking the runway.

2. Why & How It Helps (Value Proposition)

Unlocks high-concurrency parallel task execution while enforcing strict topological ordering and crash recovery.

ATOMIC FEATURE:

ArtifactPanel renderer

1. What It Does (Plain English & Analogy)

ArtifactPanel renderer is an interactive UI interface that renders html, plotly charts, code.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

ATOMIC FEATURE:

tool execution timeline

1. What It Does (Plain English & Analogy)

tool execution timeline is a foundational architectural component of  ChatView that step-by-step tool waterfall.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

suggested prompt chips

1. What It Does (Plain English & Analogy)

suggested prompt chips is a foundational architectural component of  ChatView that clickable pre-canned prompts.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

COMPONENT (UNDER 13 STUDIO VIEWS):

 webui/src/views/CanvasView.jsx

CanvasView (DAG)

1. What It Does (Plain English & Analogy)

 CanvasView (DAG) is an interactive UI interface that visual workflow pipeline builder.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

ATOMIC FEATURE:

drag-and-drop 2D board

1. What It Does (Plain English & Analogy)

drag-and-drop 2D board is a foundational architectural component of  CanvasView (DAG) that freeform visual node layout.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

4 node types (agent, tool, hitl, memory)

1. What It Does (Plain English & Analogy)

4 node types (agent, tool, hitl, memory) is a foundational architectural component of  CanvasView (DAG) that composable building blocks.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

DFS cycle detection

1. What It Does (Plain English & Analogy)

DFS cycle detection is a foundational architectural component of  CanvasView (DAG) that visual warning if cycle created.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

Bezier wire routing

1. What It Does (Plain English & Analogy)

Bezier wire routing is a foundational architectural component of  CanvasView (DAG) that smooth connecting wires.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

Kahn topological execution

1. What It Does (Plain English & Analogy)

Kahn topological execution is a foundational architectural component of  CanvasView (DAG) that executes pipeline step-by-step.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

persistent pipeline templates

1. What It Does (Plain English & Analogy)

persistent pipeline templates is a foundational architectural component of  CanvasView (DAG) that saves workflows to sqlite.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

📌 durable node checkpoints

1. What It Does (Plain English & Analogy)

durable node checkpoints is a foundational architectural component of  CanvasView (DAG) that saves outputs of each node.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

📌 crash recovery resume

1. What It Does (Plain English & Analogy)

crash recovery resume is a schema resilience utility that resumes execution from last checkpoint.



Analogy: Like an intelligent spelling auto-correct system on a smartphone keyboard that fixes minor typos seamlessly.

2. Why & How It Helps (Value Proposition)

Prevents preventable tool call crashes and runtime validation errors caused by minor LLM argument naming discrepancies.

COMPONENT (UNDER 13 STUDIO VIEWS):

 webui/src/views/ApprovalsView.jsx

📌 ApprovalsView (HITL)

1. What It Does (Plain English & Analogy)

 ApprovalsView (HITL) is an interactive UI interface that human-in-the-loop review queue.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

ATOMIC FEATURE:

pending approval queue

1. What It Does (Plain English & Analogy)

pending approval queue is a foundational architectural component of  ApprovalsView (HITL) that cards showing blocked tool calls.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

countdown timers

1. What It Does (Plain English & Analogy)

countdown timers is a foundational architectural component of  ApprovalsView (HITL) that live countdown to auto-deny.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

safety rules registry

1. What It Does (Plain English & Analogy)

safety rules registry is a foundational architectural component of  ApprovalsView (HITL) that add/remove triggers requiring approval.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

resolution audit history

1. What It Does (Plain English & Analogy)

resolution audit history is a foundational architectural component of  ApprovalsView (HITL) that log of who approved/denied what.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

COMPONENT (UNDER 13 STUDIO VIEWS):

 webui/src/views/SmartRouterView.jsx

SmartRouterView

1. What It Does (Plain English & Analogy)

 SmartRouterView is an interactive UI interface that dynamic routing playground.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

ATOMIC FEATURE:

interactive playground

1. What It Does (Plain English & Analogy)

interactive playground is a foundational architectural component of  SmartRouterView that test prompts and inspect classification.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

2-stage pipeline card

1. What It Does (Plain English & Analogy)

2-stage pipeline card is a foundational architectural component of ⚡ SmartRouterView that visualizes reason -> execute stages.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🖥️ Web UI Studio pipeline.

ATOMIC FEATURE:

threshold tuning sliders

1. What It Does (Plain English & Analogy)

threshold tuning sliders is a foundational architectural component of ⚡ SmartRouterView that adjust confidence thresholds.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🖥️ Web UI Studio pipeline.

ATOMIC FEATURE:

routing trace log inspector

1. What It Does (Plain English & Analogy)

routing trace log inspector is a foundational architectural component of ⚡ SmartRouterView that table of historical routing decisions.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🖥️ Web UI Studio pipeline.

COMPONENT (UNDER 13 STUDIO VIEWS):

webui/src/views/ToolsView.jsx

ToolsView

1. What It Does (Plain English & Analogy)

 ToolsView is an interactive UI interface that mcp tool catalog, parameter sandbox, latency meter.

 **Analogy:** Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 13 STUDIO VIEWS):

webui/src/views/SkillsView.jsx

SkillsView

1. What It Does (Plain English & Analogy)

 SkillsView is an interactive UI interface that 10 domain persona cards with custom skill creator.

 **Analogy:** Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 13 STUDIO VIEWS):

webui/src/views/WorkspaceView.jsx

WorkspaceView

1. What It Does (Plain English & Analogy)

 WorkspaceView is an interactive UI interface that in-browser file browser, code viewer & editor.

 **Analogy:** Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 13 STUDIO VIEWS):

webui/src/views/TelemetryView.jsx

TelemetryView

1. What It Does (Plain English & Analogy)

 TelemetryView is an interactive UI interface that system analytics & spend forecasting.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

ATOMIC FEATURE:

KPI overview cards

1. What It Does (Plain English & Analogy)

KPI overview cards is a foundational architectural component of  TelemetryView that total calls, total tokens, total spend.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

token distribution pie chart

1. What It Does (Plain English & Analogy)

token distribution pie chart is a foundational architectural component of  TelemetryView that prompt vs completion token share.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:**model share bar chart****1. What It Does (Plain English & Analogy)**

model share bar chart is a configured LLM endpoint that provides requests per model breakdown.



Analogy: Like a specialized vehicle in a corporate fleet—from agile compact delivery vans to heavy-duty freight trucks—chosen for the specific cargo.

2. Why & How It Helps (Value Proposition)

Allows the platform to dynamically match computational complexity to the most cost-effective and capable inference engine.

ATOMIC FEATURE:**30-day cost forecasting chart****1. What It Does (Plain English & Analogy)**

30-day cost forecasting chart is a foundational architectural component of  TelemetryView that projected monthly bill.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:**anomaly detection alerts****1. What It Does (Plain English & Analogy)**

anomaly detection alerts is a foundational architectural component of  TelemetryView that flags latency spikes.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

COMPONENT (UNDER 13 STUDIO VIEWS):

 webui/src/views/AuditLogsView.jsx



AuditLogsView

1. What It Does (Plain English & Analogy)

 AuditLogsView is an interactive UI interface that 3-tier hierarchical trace inspector.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

ATOMIC FEATURE:

Conversation -> Turn -> Request tree

1. What It Does (Plain English & Analogy)

Conversation -> Turn -> Request tree is a foundational architectural component of  AuditLogsView that nested trace hierarchy.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

flat streaming view

1. What It Does (Plain English & Analogy)

flat streaming view is a foundational architectural component of  AuditLogsView that chronological log stream.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:**live SSE auto-refresh****1. What It Does (Plain English & Analogy)**

live SSE auto-refresh is a foundational architectural component of  AuditLogsView that pushes new logs automatically.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:**CSV / JSONL export****1. What It Does (Plain English & Analogy)**

CSV / JSONL export is a foundational architectural component of  AuditLogsView that one-click download of audit logs.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

COMPONENT (UNDER 13 STUDIO VIEWS): webui/src/views/EvalsView.jsx** EvalsView****1. What It Does (Plain English & Analogy)**

 EvalsView is an interactive UI interface that single model runner, multi-model comparison, trace modal.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 13 STUDIO VIEWS):

📄 webui/src/views/OrchestratorView.jsx

🌐 OrchestratorView

1. What It Does (Plain English & Analogy)

🌐 OrchestratorView is an interactive UI interface that supervisor dag runner, sse worker stream, debate manager.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 13 STUDIO VIEWS):

📄 webui/src/views/MemoryView.jsx

🧠 MemoryView

1. What It Does (Plain English & Analogy)

🧠 MemoryView is an interactive UI interface that memory explorer and graph visualizer.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

ATOMIC FEATURE:

vector memory vault

1. What It Does (Plain English & Analogy)

vector memory vault is a foundational architectural component of 🧠 MemoryView that search and inspect stored memories.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the 🖥️ Web UI Studio pipeline.

ATOMIC FEATURE:

GraphRAG 2D visualizer

1. What It Does (Plain English & Analogy)

GraphRAG 2D visualizer is a foundational architectural component of  MemoryView that interactive force-directed entity graph.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

multi-hop pathfinder

1. What It Does (Plain English & Analogy)

multi-hop pathfinder is a foundational architectural component of  MemoryView that finds shortest link between 2 entities.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

COMPONENT (UNDER 13 STUDIO VIEWS):

 webui/src/views/SettingsView.jsx



SettingsView

1. What It Does (Plain English & Analogy)

 SettingsView is an interactive UI interface that runtime configuration and api keys.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

ATOMIC FEATURE:

transport toggle (HTTP vs Stdio)

1. What It Does (Plain English & Analogy)

transport toggle (HTTP vs Stdio) is a foundational architectural component of  SettingsView that switches gateway connection mode.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

Ollama host configuration

1. What It Does (Plain English & Analogy)

Ollama host configuration is a foundational architectural component of  SettingsView that sets local inference url.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

multi-provider API keys editor

1. What It Does (Plain English & Analogy)

multi-provider API keys editor is a foundational architectural component of  SettingsView that sets openai, anthropic, gemini keys.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

ATOMIC FEATURE:

system telemetry gauges

1. What It Does (Plain English & Analogy)

system telemetry gauges is a foundational architectural component of  SettingsView that cpu, ram, and disk live gauges.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

SUB HEADER (OF WEB UI STUDIO):

 webui/src/components/

7 Shared Components

1. What It Does (Plain English & Analogy)

7 Shared Components is a foundational architectural component of  Web UI Studio that reusable ui components across all views.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.

COMPONENT (UNDER 7 SHARED COMPONENTS):

Sidebar (5 navigation groups)

1. What It Does (Plain English & Analogy)

Sidebar (5 navigation groups) is an interactive UI interface that chat, automation, knowledge, observability, settings.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 7 SHARED COMPONENTS):**TopHeader with status indicators****1. What It Does (Plain English & Analogy)**

TopHeader with status indicators is an interactive UI interface that gateway health, active model, token budget.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 7 SHARED COMPONENTS):**InspectorModal (execution waterfall)****1. What It Does (Plain English & Analogy)**

InspectorModal (execution waterfall) is an interactive UI interface that detailed breakdown of single request.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 7 SHARED COMPONENTS):**HITLApprovalModal (risk badge + timer)****1. What It Does (Plain English & Analogy)**

HITLApprovalModal (risk badge + timer) is an interactive UI interface that popup for approving high-risk tool call.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 7 SHARED COMPONENTS):

CreateSkillModal

1. What It Does (Plain English & Analogy)

CreateSkillModal is an interactive UI interface that form to author new domain personas.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 7 SHARED COMPONENTS):

ArtifactPanel (HTML / Plotly / Code)

1. What It Does (Plain English & Analogy)

ArtifactPanel (HTML / Plotly / Code) is an interactive UI interface that side drawer for rich interactive assets.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

COMPONENT (UNDER 7 SHARED COMPONENTS):

EvalTraceModal (4-tier coordinates)

1. What It Does (Plain English & Analogy)

EvalTraceModal (4-tier coordinates) is an interactive UI interface that granular inspector for evaluation runs.



Analogy: Like a dedicated diagnostic monitor in a hospital operating room displaying vital patient signals clearly to the medical team.

2. Why & How It Helps (Value Proposition)

Provides operators with intuitive visual controls and live telemetry, eliminating the need to inspect raw database rows or console logs.

SUB HEADER (OF WEB UI STUDIO):

40+ API Client Methods

1. What It Does (Plain English & Analogy)

40+ API Client Methods is a foundational architectural component of  Web UI Studio that type-safe javascript client for all backend endpoints.



Analogy: Like a dedicated instrument gauge on a flight deck providing reliable, real-time measurements for navigation.

2. Why & How It Helps (Value Proposition)

Ensures predictable behavior, enhances system maintainability, and guarantees seamless integration across the  Web UI Studio pipeline.